



<https://vrgl.ir/W9GDt>

ویرگول - جایی برای نوشتن و خواندن

هوش مصنوعی واقعاً چیست و مدل چگونه یاد می‌گیرد؟

0. مقدمه

من به عنوان مهندس و معمار نرم افزار، در مسیری قرار گرفته‌ام که طراحی و تولید محصولات مبتنی بر هوش مصنوعی بخشی از مسئولیت حرفه‌ای من است. در چنین مسیری خیلی زود روشن می‌شود که آشنایی سطحی با نام ابزارها، مدل‌ها و چارچوب‌ها برای تصمیم‌گیری معماری کافی نیست. اگر قرار باشد درباره مدل‌های زبانی بزرگ، بازیابی، عامل‌های هوشمند، سامانه‌های چندعاملی، امنیت و معماری سازمانی هوش مصنوعی تصمیمی جدی گرفته شود، باید از لایه‌ای بسیار بنیادی‌تر آغاز کرد: هوش مصنوعی چیست، یادگیری ماشین چه تفاوتی با برنامه‌نویسی کلاسیک دارد، مدل دقیقاً چیست، پارامتر چه نقشی دارد و جمله «مدل یاد گرفته است» در سطح محاسباتی چه معنایی دارد.

این نوشته حاصل تلاش من برای فهم منسجم همین پایه‌هاست. هدف این نیست که خواننده مجموعه‌ای از تعریف‌های کوتاه را حفظ کند؛ هدف این است که بتواند منطق درونی مفاهیم را دنبال کند و بفهمد چرا هر مفهوم به مفهوم بعدی منتهی می‌شود.

متن برای خواننده‌ای نوشته شده است که مهندسی نرم افزار را می‌شناسد اما قرار نیست از قبل دانش یادگیری ماشین داشته باشد. به همین دلیل از مفاهیم آشنایی مانند تابع، حالت، رابط، زمان اجرا، استقرار، پایگاه داده، قواعد کسب و کار و پایش به عنوان پل استفاده شده است. در عین حال هر جا این تشبیه‌ها محدودیت دارند، محدودیت آن‌ها نیز توضیح داده شده تا مدل ذهنی نادرستی ساخته نشود.

در متن تا جای ممکن از برابره‌ای فارسی استفاده شده است. هدف این است که نثر، فارسی و روان باقی بماند و خواننده برای دنبال کردن یک جمله ناچار نباشد میان تعداد زیادی واژه انگلیسی جابه‌جا شود. در پایان، واژه‌نامه‌ای فارسی به انگلیسی آمده است تا اصطلاح بین‌المللی هر مفهوم نیز در دسترس باشد.

اگر در متن اشکال، ابهام یا نکته‌ای برای بهبود مشاهده کردید، یا پرسشی درباره محتوای این نوشته داشتید، می‌توانید از طریق نشانی ar.roumand@gmail.com با من در ارتباط باشید.

1. از هوش مصنوعی تا جایی که نرم افزار دیگر فقط از قواعد ساخته نمی شود

سؤال مرکزی این نوشته را از همان ابتدا می توان چنین مطرح کرد: وقتی گفته می شود «یک ماشین چیزی را یاد گرفته است»، دقیقاً چه چیزی در ماشین عوض شده است؟ این سؤال در نگاه نخست ساده به نظر می رسد، اما اگر پاسخ روشنی برای آن نداشته باشیم، بسیاری از مفاهیم بعدی هوش مصنوعی به مجموعه ای از اصطلاحات تبدیل می شوند که ارتباط واقعی میان آن ها را نمی بینیم. ممکن است بدانیم مدل زبانی بزرگ چیست، درباره شبکه عصبی یا ترنسفورمر صحبت کنیم و حتی یک محصول مبتنی بر هوش مصنوعی بسازیم، اما هنوز ندانیم چه تفاوت بنیادی میان نرم افزاری که رفتار آن را انسان نوشته و نرم افزاری که بخشی از رفتار خود را از داده آموخته وجود دارد.

برای رسیدن به پاسخ، لازم نیست از مدل های زبانی یا شبکه های عصبی آغاز کنیم. نقطه شروع مناسب برای یک مهندس نرم افزار، همان دنیایی است که از قبل می شناسد: نرم افزاری که قواعد رفتاری آن به صورت صریح نوشته می شوند. فرض کنید قرار است برای یک بانک، سامانه ای برای شناسایی تراکنش مشکوک طراحی شود. هنوز هیچ یادگیری ماشینی در کار نیست. متخصصان کشف تقلب مجموعه ای از الگوهای شناخته شده را در اختیار تیم توسعه می گذارند و تیم این دانش را به منطق قابل اجرا تبدیل می کند.

ممکن است یکی از قواعد بگوید اگر مبلغ تراکنش از حدی مشخص بیشتر بود و کشور مقصد با الگوی معمول مشتری تفاوت داشت، امتیاز ریسک افزایش پیدا کند. قاعده ی دیگری ممکن است بگوید اگر در فاصله زمانی کوتاهی دو تراکنش از دو نقطه جغرافیایی بسیار دور ثبت شده باشد، احتمال سوءاستفاده بیشتر است. قاعده دیگری تعداد تلاش های ناموفق اخیر، نوع دستگاه یا وضعیت پذیرنده را در نظر می گیرد.

- 1 مبلغ_تراکنش < آستانه_مبلغ_بالا و کشور_مقصد در کشور_های_معمول_مشتری نباشد
- 2 امتیاز_ریسک += ۳۰
- 3
- 4 : اگر سفر_ناممکن_تشخیص_داده شد
- 5 امتیاز_ریسک += ۵۰
- 6
- 7 : اگر تعداد_تلاش_ناموفق_یک_ساعت_اخیر < ۵
- 8 امتیاز_ریسک += ۲۰
- 9
- 10 : اگر امتیاز_ریسک <= آستانه_بازبینی
- 11 () ارسال_برای_بازبینی_انسانی

این مثال عمده ساده است. یک موتور قواعد واقعی می‌تواند صدها یا هزاران قاعده، استثنا، جدول تصمیم، اولویت و سیاست داشته باشد. با این حال یک ویژگی بنیادی همچنان ثابت می‌ماند: منطق تصمیم‌گیری به شکلی صریح توسط انسان وارد سامانه شده است. اگر از تیم پرسیده شود «چرا این تراکنش علامت‌گذاری شد؟»، دست‌کم از نظر تئوری می‌توان مسیر اجرای قواعد را دنبال کرد و توضیح داد کدام قاعده فعال شده، چه امتیازی اضافه کرده و چگونه جمع امتیاز از آستانه عبور کرده است.

دانش مورد استفاده سامانه در چنین حالتی در کد منبع، موتور قواعد، پیکربندی یا جدول تصمیم قابل مشاهده است. این همان جهان آشنای مهندسی نرم‌افزار کلاسیک است: مسئله تحلیل می‌شود، رفتار مطلوب به الگوریتم و قاعده تبدیل می‌شود، آن منطق پیاده‌سازی می‌شود و در زمان اجرا روی داده ورودی اعمال می‌گردد.

نکته مهم این است که چنین سامانه‌ای می‌تواند بسیار پیچیده و حتی در معنای تاریخی «هوشمند» باشد، بدون اینکه چیزی از داده یاد گرفته باشد. این نکته ما را به نخستین تفکیک اساسی می‌رساند.

1.1. هوش مصنوعی گسترده‌تر از یادگیری ماشین است

امروزه در گفت‌وگوی روزمره وقتی از هوش مصنوعی سخن گفته می‌شود، اغلب منظور یادگیری ماشین، هوش مصنوعی مولد یا حتی مشخصاً مدل‌های زبانی بزرگ است. این استفاده قابل درک است، زیرا موج فعلی هوش مصنوعی عمدتاً با یادگیری عمیق و مدل‌های مولد شناخته می‌شود. با این حال، از نظر علمی و تاریخی، هوش مصنوعی مفهومی گسترده‌تر است.

هوش مصنوعی را می‌توان حوزه‌ای علمی و مهندسی دانست که می‌پرسد چگونه می‌توان سامانه‌هایی ساخت که بتوانند وظایفی مانند استدلال، تصمیم‌گیری، برنامه‌ریزی، حل مسئله، ادراک، یادگیری یا کار با زبان را انجام دهند. در این تعریف هیچ الزامی وجود ندارد که سیستم برای انجام وظیفه حتماً از داده آموزش دیده باشد.

برای نمونه، یک برنامه شطرنج می‌تواند با جست‌وجو در فضای حالت‌ها و استفاده از الگوریتم‌های تصمیم‌گیری حرکت بعدی را انتخاب کند. یک سامانه برنامه‌ریزی می‌تواند با مجموعه‌ای از وضعیت‌ها، کنش‌ها و محدودیت‌ها یک برنامه اجرایی بسازد. یک سامانه خبره پزشکی می‌تواند مجموعه بزرگی از قواعد تخصصی را اعمال کند. همه این‌ها می‌توانند در قلمرو هوش مصنوعی قرار گیرند، در حالی که یادگیری ماشین نیستند.

البته مرز دقیق اینکه کدام نرم افزار را باید هوش مصنوعی نامید همیشه کاملاً ریاضی و صلب نیست. وجود یک شرط ساده مانند «اگر مبلغ بیش از یک میلیون بود، تأیید مدیر لازم است» به تنهایی یک سامانه را هوش مصنوعی نمی کند. اما سامانه ای که دانش تخصصی گسترده، نمایش نمادین مفاهیم و سازوکار استنتاج دارد، از نظر تاریخی در خانواده هوش مصنوعی نمادین قرار می گیرد.

در هوش مصنوعی نمادین تلاش می شود دانش به صورت صریح بازنمایی شود. اگر سامانه بداند «همه پستانداران خون گرم اند» و «نهنگ یک پستاندار است»، می تواند نتیجه بگیرد «نهنگ خون گرم است». رابطه ها، نمادها و قواعد در خود سامانه حضور دارند و موتور استنتاج روی آن ها عمل می کند. یادگیری ماشین از مسیر دیگری وارد مسئله می شود.

1.2. جایی که نوشتن قواعد دشوار می شود

حال دوباره به تشخیص تقلب بازگردیم. تصور کنید پس از چند سال توسعه، تیم متوجه شده است که الگوهای تقلب دائماً تغییر می کنند و بسیاری از آن ها با یک یا دو شرط ساده قابل شناسایی نیستند. ممکن است یک تراکنش از نظر مبلغ عادی باشد، کشور مقصد نیز عادی باشد و دستگاه هم قبلاً دیده شده باشد، اما ترکیب ظریفی از زمان تراکنش، فاصله از تراکنش قبلی، نوع پذیرنده، الگوی تاریخی مشتری، سرعت تغییر موقعیت جغرافیایی و چند عامل دیگر رفتاری غیرعادی ایجاد کند.

در چنین حالتی مشکل فقط این نیست که تعداد قواعد زیاد می شود. مشکل عمیق تر آن است که حتی متخصص ممکن است نتواند دانش ضمنی خود را به مجموعه ای کامل از شرط های صریح تبدیل کند. ممکن است با دیدن یک مورد بگوید «این تراکنش مشکوک به نظر می رسد»، اما نتواند همین قضاوت را به چهل شرط دقیق و قابل نگهداری تبدیل کند.

این مسئله در حوزه های دیگر نیز دیده می شود. انسان به سادگی یک گربه را از سگ تشخیص می دهد، اما نوشتن قواعدی که از سطح پیکسل ها هر گربه را در هر زاویه، نور، اندازه و پس زمینه ای تشخیص دهد بسیار دشوار است. در تشخیص هرزنامه نیز می توان بر اساس واژه ها، تعداد پیوندها و دامنه فرستنده قواعدی نوشت، اما فرستندگان هرزنامه دائماً رفتار خود را تغییر می دهند و ایمیل های سالم نیز ممکن است همان نشانه ها را داشته باشند.

- | | |
|---|--|
| 1 | : اگر عنوان شامل «جایزه رایگان» باشد |
| 2 | امتیاز_هرزنامه += ۲۰ |
| 3 | |
| 4 | : اگر تعداد_پیوندها < ۵ باشد |
| 5 | امتیاز_هرزنامه += ۱۵ |
| 6 | |
| 7 | : اگر دامنه_فرستنده در فهرست_سیاه باشد |
| 8 | امتیاز_هرزنامه += ۵۰ |

یادگیری ماشین در چنین مسائلی تغییر مهمی در روش ساخت نرم افزار ایجاد می کند. در برنامه نویسی کلاسیک، توسعه دهنده تلاش می کند رابطه میان ورودی و خروجی را به صورت صریح در برنامه قرار دهد. به شکل ساده می توان این وضعیت را چنین نوشت:

داده + قواعد → خروجی

در یک نوع مهم از یادگیری ماشین، یعنی یادگیری نظارت شده، نمونه های ورودی و خروجی های مورد انتظار در اختیار فرایند آموزش قرار می گیرند و فرایند آموزش تلاش می کند پارامترهای مدل را به شکلی تنظیم کند که رابطه مفیدی میان ورودی ها و خروجی ها ایجاد شود:

داده + خروجی های مورد انتظار → فرایند آموزش → مدل

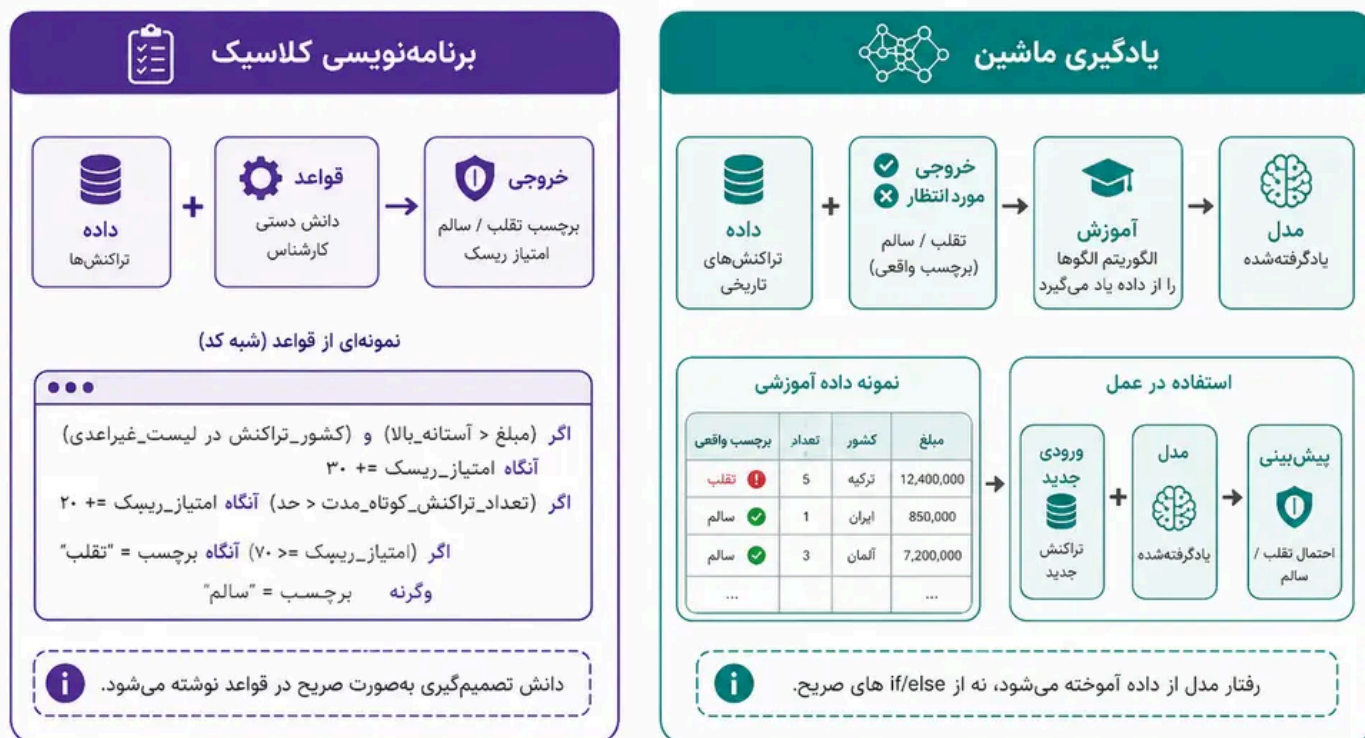
سپس مدل آموزش دیده برای ورودی جدید استفاده می شود:

ورودی جدید + مدل → پیش بینی

این دو رابطه به تنهایی تعریف کامل برنامه نویسی و یادگیری ماشین نیستند؛ هدف آن ها نشان دادن محل شکل گیری بخشی از رفتار است. در برنامه نویسی کلاسیک قواعد اصلی توسط انسان نوشته شده اند. در یادگیری ماشین بخشی از رفتار از طریق تنظیم پارامترهای مدل با استفاده از داده شکل می گیرد.

— دو رویکرد برای ساخت سیستم هوشمند —

مثال: تشخیص تقلب در تراکنش‌های مالی



یادگیری ماشین جایگزین کامل نرم‌افزار قطعی نیست؛

سامانه واقعی معمولاً ترکیبی از هر دو است.

1.3. چرا جمله‌ی «مدل خودش قواعد را پیدا می‌کند» کامل نیست؟

یک توضیح رایج می‌گوید «در برنامه‌نویسی قواعد را انسان می‌نویسد، اما در یادگیری ماشین کامپیوتر خودش قواعد را پیدا می‌کند». این جمله برای ایجاد شهود اولیه بد نیست، اما اگر آن را دقیق تلقی کنیم گمراه‌کننده است. بسیاری از مدل‌ها، به‌ویژه شبکه‌های عصبی، پس از آموزش فهرستی از قواعد قابل خواندن مانند شرط‌های یک موتور قواعد تولید نمی‌کنند.

آنچه معمولاً یاد گرفته می‌شود مجموعه‌ای از پارامترهای عددی است که در یک ساختار محاسباتی قرار گرفته‌اند. وقتی این پارامترها تغییر می‌کنند، رفتار تابع نیز تغییر می‌کند. بنابراین بهتر است به جای «استخراج قواعد» بگوییم «تنظیم پارامترهای یک مدل برای ایجاد رفتار مطلوب‌تر نسبت به داده و هدف آموزش».

در برخی مدل‌ها مانند درخت تصمیم خروجی واقعاً می‌تواند تا حدی شبیه مجموعه قواعد باشد، اما این ویژگی را نباید به کل یادگیری ماشین تعمیم داد.

1.4. چرا یادگیری ماشین جای برنامه‌نویسی کلاسیک را نمی‌گیرد؟

فرض کنید در یک سازمان بزرگ درخواست‌های پشتیبانی باید به تیم‌های پایگاه داده، شبکه، امنیت، برنامه کاربردی یا محل کار دیجیتال هدایت شوند. نسخه اولیه سیستم ممکن است بر اساس واژه‌های کلیدی کار کند: اگر متن شامل نام یک پایگاه داده بود، درخواست به تیم پایگاه داده برود؛ اگر شامل شبکه خصوصی مجازی بود، به تیم شبکه هدایت شود. اما یک درخواست واقعی می‌تواند بنویسد «گزارش پایان روز از دیشب بعضی وقت‌ها باز می‌شود و بعضی وقت‌ها زمانش تمام می‌شود، ولی سرویس اصلی ظاهراً سالم است». ممکن است هیچ واژه کلیدی مستقیمی وجود نداشته باشد، در حالی که تاریخچه درخواست‌ها نشان دهد چنین الگوهایی معمولاً به تیم خاصی مربوط‌اند.

یک مدل دسته‌بندی می‌تواند با استفاده از هزاران درخواست قبلی احتمال تعلق درخواست جدید به هر تیم را برآورد کند.

خروجی مدل برای یک درخواست: پایگاه داده: ۰/۶۷ برنامه کاربردی: ۰/۲۱ شبکه: ۰/۰۸ سایر: ۰/۰۴

با این حال، ممکن است سیاست سازمان بگوید هر درخواستی که احتمال نشت اطلاعات یا رخداد امنیتی دارد باید بدون توجه به مدل مستقیماً وارد فرایند امنیت شود. این سیاست باید صریح و تعیین‌گرا باقی بماند. در نتیجه معماری واقعی ترکیبی از قواعد قطعی، مدل آماری، سیاست کسب‌وکار، کنترل انسانی و نرم‌افزار کلاسیک است.

احراز هویت، مجوز، تراکنش پایگاه داده، محدودیت‌های مالی، ثبت ممیزی، پیام‌رسانی، مدیریت خطا، نرخ‌دهی و گردش کار از بین نمی‌روند. مدل فقط یکی از مؤلفه‌های سیستم است. هرچا رفتار صحیح به‌صورت دقیق و پایدار قابل بیان است، قاعده صریح اغلب انتخاب بهتری است. یادگیری ماشین زمانی ارزش ویژه دارد که رابطه میان ورودی و خروجی آن‌قدر پیچیده، آماری یا پنهان باشد که نوشتن مجموعه قواعد نگهداری‌پذیر بسیار دشوار شود.

1.5. جایگاه یادگیری عمیق، هوش مصنوعی مولد، مدل پایه و مدل زبانی بزرگ

یادگیری عمیق یکی از خانواده‌های یادگیری ماشین است که عمدتاً از شبکه‌های عصبی چندلایه استفاده می‌کند. بنابراین رابطه اصلی چنین است:

یادگیری عمیق → یادگیری ماشین → هوش مصنوعی

روش‌هایی مانند رگرسیون خطی، رگرسیون لجستیک، درخت تصمیم و جنگل تصادفی می‌توانند یادگیری ماشین باشند بدون آنکه یادگیری عمیق باشند. در مقابل، هوش مصنوعی مولد درباره نوع قابلیت خروجی صحبت می‌کند: سامانه می‌تواند متن، تصویر، صوت، ویدئو یا کد تولید کند. هوش مصنوعی مولد با مدل زبانی بزرگ یکی نیست؛ تولید تصویر نیز هوش مصنوعی مولد است.

مدل پایه معمولاً مدلی است که در مقیاس بزرگ و روی داده گسترده از پیش آموزش داده شده و می‌تواند پایه تعداد زیادی کاربرد باشد. مدل زبانی بزرگ نیز مدلی بزرگ برای کار با زبان است. بسیاری از مدل‌های زبانی بزرگ مدرن مدل پایه‌اند، اما هر مدل پایه الزاماً مدل زبانی نیست؛ می‌تواند دیداری یا چندوجهی باشد.



1.6. اثر نخستین این تغییر بر معماری

در نرم‌افزار کلاسیک معمولاً تصور می‌کنیم رفتار سامانه عمدتاً از کد و پیکربندی ناشی می‌شود. در یادگیری ماشین این فرض کامل نیست. ممکن است دو تیم دقیقاً کد آموزش، معماری مدل و پیکربندی یکسان داشته باشند، اما یکی روی داده بانک الف و دیگری روی داده بانک ب آموزش دهد و در نهایت دو مدل با رفتار متفاوت تولید شود.

حتی ممکن است کد سرویس تولید در دو محیط دقیقاً یکسان باشد ولی نسخه مدل متفاوتی بارگذاری شده باشد. در این صورت رفتار تولید تغییر می‌کند بدون اینکه تغییر کد برنامه وجود داشته باشد. از این‌جا نسخه مدل و نسخه داده نیز وارد زنجیره ردیابی می‌شوند.

اگر در تولید رفتار اشتباهی دیده شود، دیگر فقط نمی‌پرسیم کدام تغییر کد باعث مشکل شده است. ممکن است داده آموزشی تغییر کرده باشد، برچسب‌ها مشکل داشته باشند، توزیع داده تولید عوض شده باشد، مدل جدیدی مستقر شده باشد یا آستانه تصمیم‌گیری تغییر کرده باشد. این نخستین نشانه از تفاوت معماری سامانه‌های یادگیری ماشین با سامانه‌های کاملاً تعیین‌گراست.

2. مدل چیست و یادگیری در ساده‌ترین شکل چه اتفاقی است؟

اکنون به پرسشی می‌رسیم که بخش نخست با آن پایان یافت: اگر رفتار یادگیری ماشین به صورت مجموعه قواعد صریح در کد نوشته نشده است، این رفتار دقیقاً کجا قرار دارد؟ برای پاسخ، بهترین راه آن است که مدل را در ابتدا مانند یک تابع در نظر بگیریم.

در مهندسی نرم‌افزار هر روز با تابع سروکار داریم. فرض کنید تابعی برای محاسبه مبلغ نهایی سفارش وجود دارد. اگر پیاده‌سازی تابع و سفارش مشخص باشند، می‌توانیم مسیر محاسبه را دنبال کنیم: قیمت اقلام جمع می‌شود، تخفیف اعمال می‌شود، مالیات افزوده می‌شود و هزینه ارسال در نظر گرفته می‌شود. جزئیات رفتار را برنامه‌نویس تعریف کرده است.

برای مدل نیز می‌توان در ساده‌ترین سطح نوشت:

خروجی = مدل(ورودی)

برای تشخیص تقلب می‌توان نوشت «نمره تقلب = مدل(تراکنش)»، برای تصویر «کلاس تصویر = مدل(تصویر)» و برای قیمت خانه «قیمت پیش‌بینی‌شده = مدل(ویژگی‌های خانه)». این تشبیه مفید است زیرا مدل در زمان استنتاج واقعاً محاسباتی انجام می‌دهد که ورودی را به خروجی تبدیل می‌کنند. تفاوت بنیادی آن است که همه جزئیات رفتار مدل مستقیماً توسط برنامه‌نویس نوشته نشده‌اند.

2.1. ساخت یک مدل خطی از صفر

فرض کنید یک شرکت املاک اطلاعات فروش خانه‌ها را در اختیار دارد. برای ساده‌سازی، تنها ویژگی مورد استفاده متراف است. داده آموزشی مصنوعی زیر را در نظر بگیرید. واحد قیمت میلیارد تومان است.

متراف به قیمت واقعی

پرسش این است: اگر فردا خانه‌ای ۱۰۰ متری مشاهده شود، چگونه می‌توان قیمت آن را تخمین زد؟ یک توسعه‌دهنده می‌تواند داده‌ها را نگاه کند و خودش رابطه را استخراج کند، اما در این صورت دانش اصلی را انسان نوشته است. هدف ما این است که ماشین مقادیر مناسب رابطه را از داده پیدا کند.

نخست باید مشخص کنیم مدل اجازه دارد چه نوع رابطه‌ای را نمایش دهد. در این مثال فرض می‌کنیم رابطه خطی است:

$$\hat{y} = wx + b$$

در این فرمول x ورودی یا ویژگی است. اگر خانه ۹۰ متر باشد، x برابر ۹۰ است. y قیمت واقعی ثبت شده در داده است. علامت $\hat{y}$ یا «وای هت» مقدار پیش‌بینی شده مدل را نشان می‌دهد. تفاوت میان y و $\hat{y}$ بعداً برای اندازه‌گیری خطا استفاده می‌شود.

2.2. w و b چه معنایی دارند؟

w ضریبی است که تعیین می‌کند تغییر x با چه شدت و جهتی بر خروجی اثر بگذارد. b جمله ثابتی است که اجازه می‌دهد خط مجبور نباشد از مبدأ عبور کند. اگر w برابر ۰/۰۵ و b برابر ۱/۵ باشد، مدل چنین است:

$$\hat{y} = 0.05x + 1.5$$

برای خانه ۱۰۰ متری محاسبه می‌کنیم:

$$\hat{y} = 0.05 \times 100 + 1.5 = 6.5$$

مدل قیمت را ۶/۵ میلیارد تخمین می‌زند. اما پرسش اصلی این است که چرا w باید ۰/۰۵ و b باید ۱/۵ باشد؟ اگر این اعداد را انسان با حدس انتخاب کرده باشد، هنوز آموزش رخ نداده است. یادگیری زمانی اتفاق می‌افتد که این مقادیر با استفاده از داده پیدا شوند.

مدل خطی و مثال قیمت خانه



مدل خطی (ساده‌ترین مدل)

رابطه‌ی بین متراژ خانه و قیمت را با یک خط راست تقریب می‌زنند.



داده‌های نمونه (خانه‌های فروش‌رفته)

متراژ (متر مربع) x	قیمت (میلیارد تومان) $\hat{y}$
50	4
70	5
90	6
110	7
130	8



فرمول مدل خطی

$$\hat{y} = wx + b$$

x : متراژ خانه

$\hat{y}$: قیمت پیش‌بینی‌شده

w : شیب خط (تغییر قیمت به ازای هر متر مربع)

b : مقدار پایه (قیمت در متراژ صفر)



نمودار داده‌ها و خط مدل

قیمت
(میلیارد تومان)



مثال محاسبه

برای خانه 100 متری:

$$\hat{y} = 0.05 \times 100 + 1.5 = 6.5 \rightarrow \text{قیمت پیش‌بینی‌شده: 6.5 میلیارد تومان}$$



مدل خطی، ساده‌ترین مدل ممکن برای یادگیری مفاهیم پایه یادگیری ماشین است.

2.3. پارامتر مدل چیست؟

w و b پارامترهای مدل هستند، زیرا مقادیر عددی قابل یادگیری‌اند که بخشی از رفتار تابع را تعیین می‌کنند. اگر w و b تغییر کنند، ساختار کلی فرمول همان است اما خروجی برای ورودی یکسان عوض می‌شود. برای نمونه اگر $w = 0.01$ و $b = 1$ باشد، خانه صد متری ۲ میلیارد پیش‌بینی می‌شود. اگر $w = 0.1$ و $b = 1$ باشد، همان خانه ۱۱ میلیارد پیش‌بینی می‌شود.

به همین دلیل تعریف دقیق و عملی پارامتر این است: مقداری عددی در مدل که بخشی از رفتار مدل را تعیین می‌کند و در فرایند آموزش با استفاده از داده تنظیم می‌شود.

برای یک مهندس نرم‌افزار یک ابهام واژگانی رایج وجود دارد. در برنامه‌نویسی ممکن است آرگومان تابع را نیز پارامتر بنامیم، اما در یادگیری ماشین x را «پارامتر مدل» نمی‌نامیم. x ورودی است و به ازای هر نمونه تغییر می‌کند؛ w و b بخشی از مدل آموزش‌دیده‌اند و در استنتاج معمولاً ثابت می‌مانند.

2.4. پارامتر با پیکربندی چه تفاوتی دارد؟

اعدادی مانند زمان انتظار، تعداد تلاش مجدد یا سقف مبلغ تراکنش نیز رفتار نرم افزار را تغییر می دهند و از این نظر به پارامتر شبیه اند. اما پیکربندی معمولاً توسط انسان یا سیاست عملیاتی تعیین می شود. پارامترهای مدل عمدتاً از طریق آموزش و داده به دست می آیند. انسان میلیاردها وزن یک مدل بزرگ را به صورت دستی تنظیم نمی کند.

2.5. مدل به عنوان تابع پارامتردار

برای نمایش عمومی تر، از نماد θ برای مجموعه تمام پارامترهای مدل استفاده می شود:

$$\hat{y} = f_{\theta}(x)$$

در مدل خطی θ شامل w و b است. در یک شبکه عصبی بسیار بزرگ ممکن است θ شامل میلیاردها مقدار باشد. این نمادگذاری به ما اجازه می دهد بدون نوشتن تک تک وزن ها درباره رفتار مدل صحبت کنیم.

2.6. معماری مدل، الگوریتم آموزش و مدل آموزش دیده

اینکه تصمیم گرفته ایم رابطه از شکل $wx+b$ باشد، چیزی نیست که داده یاد گرفته باشد. ما از قبل خانواده ای از توابع خطی را انتخاب کرده ایم. آموزش در میان این خانواده دنبال مقادیر مناسب w و b می گردد. به بیان شهودی، معماری یا خانواده مدل می گوید «پاسخ باید یک خط باشد» و آموزش پاسخ می دهد «کدام خط؟».

الگوریتم آموزش نیز خود مدل نیست. الگوریتم آموزش روشی است که با استفاده از داده و یک هدف عددی پارامترهای مناسب را پیدا می کند. نزول گرادیانی یکی از روش های شناخته شده برای این کار است. در مقابل، نرخ یادگیری نمونه ای از ابرپارامتر است؛ ابرپارامترها (Hyperparameters) مقادیری هستند که نحوه آموزش یا ساختار مدل را کنترل می کنند و معمولاً خود آموزش آن ها را مثل وزن ها یاد نمی گیرد. مثلاً:

نرخ یادگیری (Learning Rate)

اندازه دسته (Batch Size)

تعداد دوره های آموزش (Epochs)

ضریب منظم سازی یا Weight Decay

تعداد لایه ها یا تعداد واحدهای بعضی لایه ها

برخی تنظیمات بهینه ساز

LearningRate=0.001

این عدد تعیین می‌کند الگوریتم آموزش با چه اندازه گامی وزن‌ها را تغییر دهد، اما خودش یکی از وزن‌های مدل نیست. یک دسته سوم هم داریم که مهم است: **متغیرهای داخلی فرایند آموزش** که نه پارامتر مدل‌اند و نه الزاماً ابرپارامتر. مثلاً شماره Batch فعلی، مقدار Loss این مرحله، Gradient‌های محاسبه‌شده، یا state داخلی بهینه‌سازی مانند momentum. این‌ها فقط متغیرهای محاسباتی یا state آموزش هستند.

پس مدل ذهنی مناسب این است:

Model Parameters

چیزهایی که یاد گرفته می‌شوند.

Hyperparameters

چیزهایی که فرایند یادگیری را تنظیم می‌کنند.

و:

Training State / Variables

مقادیر موقتی یا داخلی که هنگام اجرای آموزش استفاده می‌شوند.

فرض کنید مدل از $w = 0$ و $b = 0$ شروع کند. در این صورت:

$$\hat{y} = 0x + 0 = 0$$

این مدل برای هر خانه‌ای صفر پیش‌بینی می‌کند. برای 50 متر در حالی که مقدار واقعی 4 است صفر می‌گوید؛ برای 90 متر در حالی که مقدار واقعی 6 است باز هم صفر می‌گوید؛ و برای 130 متر در حالی که مقدار واقعی 8 است همچنان صفر می‌گوید. پس رفتار بسیار بدی دارد، اما ساختار مدل خراب نیست. فقط پارامترهای مناسب هنوز پیدا نشده‌اند.

2.8. چگونه خوب یا بد بودن پارامترها را اندازه می‌گیریم؟

برای بهتر شدن مدل باید بتوانیم کیفیت پیش‌بینی‌ها را به عدد تبدیل کنیم. در رگرسیون یکی از معیارهای ساده میانگین مجذور خطاست:

$$MSE = (1/n) \sum (\hat{y}_i - y_i)^2$$

برای مثال اگر سه مقدار واقعی 5 ، 7 و 9 باشند و مدل اولیه برای هر سه صفر پیش‌بینی کند، خطاها به ترتیب -5 ، -7 و -9 هستند. مربع آن‌ها 25 ، 49 و 81 می‌شود و میانگین تقریباً $51/67$ است. این عدد بزرگ نشان می‌دهد پارامترها نامناسب‌اند.

حال مدل دیگری با $w = 0.05$ و $b = 2$ را در نظر بگیرید. برای خانه‌های 50 ، 75 و 100 متری به ترتیب $4/5$ ، $5/75$ و 7 پیش‌بینی می‌کند. اگر مقادیر واقعی 5 ، 7 و 9 باشند، خطا بسیار کمتر از مدل صفر است و میانگین مجذور خطا حدود $1/94$ می‌شود. بنابراین این مجموعه پارامتر بهتر است.

در مجموعه داده مصنوعی مرتب خودمان، $w = 0.05$ و $b = 1.5$ حتی همه نقاط را دقیقاً توضیح می‌دهد:

$50 \rightarrow 4$ $70 \rightarrow 5$ $90 \rightarrow 6$ $110 \rightarrow 7$ $130 \rightarrow 8$

در این مثال آموزشی زیان روی داده آموزش صفر است. البته در داده واقعی معمولاً چنین نظم کاملی وجود ندارد و صفر شدن زیان آموزش نه هدف همیشگی است و نه تضمین کیفیت. همین موضوع بعداً ما را به بیش‌برازش می‌رساند.

2.9. «مدل یاد گرفت» در دو سطح

از بیرون می‌گوییم مدل رابطه متراژ و قیمت را یاد گرفته است؛ زیرا برای ورودی‌های جدید پیش‌بینی‌های معقول‌تری تولید می‌کند. در سطح محاسباتی اتفاق بسیار مشخص‌تری رخ داده است: پارامترها از حالت اولیه به مقادیر دیگری تغییر کرده‌اند.

$$w: 0 \rightarrow 0.05$$

$$b: 0 \rightarrow 1.5$$

هیچ متغیری با نام «دانش قیمت خانه» ایجاد نشده و هیچ قاعده متنی جدیدی به کد اضافه نشده است. تغییر عددی‌های داخل مدل باعث تغییر رفتار تابع شده است. همین نکته ساده پایه فهم یادگیری در شبکه‌های بسیار بزرگ‌تر است.

2.10. مدل پایگاه داده یا فهرست قواعد نیست

اگر خانه صد متری در مجموعه آموزش وجود نداشته باشد، مدل همچنان می‌تواند قیمت آن را محاسبه کند، زیرا تابع آموخته شده دارد. اگر مسئله فقط جست‌وجوی رکورد در پایگاه داده بود، نبود رکورد صد متری می‌توانست به نبود پاسخ منجر شود. مدل به جای جست‌وجوی یک مقدار ذخیره شده، محاسبه انجام می‌دهد.

$$\hat{y} = 0.05 \times 100 + 1.5 = 6.5$$

این تفاوت نخستین مقدمه برای مفهوم تعمیم است. البته مدل‌های بزرگ می‌توانند بخشی از داده آموزشی را حفظ کنند؛ بنابراین درست نیست بگوییم مدل هرگز چیزی را حفظ نمی‌کند. نکته آن است که تصور مدل به عنوان یک پایگاه داده کلید-مقدار مدل ذهنی مناسبی نیست.

2.11. داده واقعی چرا به رابطه دقیق تبدیل نمی‌شود؟

متراژ-قیمت واقعی

$$50 \rightarrow 4.2, 70 \rightarrow 5.8, 90 \rightarrow 5.5, 110 \rightarrow 8.1, 130 \rightarrow 7.6$$

در داده واقعی‌تر، قیمت فقط تابع مترایز نیست. خانه ۷۰ متری ممکن است به دلیل منطقه بهتر از خانه ۹۰ متری گران‌تر باشد. بنابراین یک خط واحد نمی‌تواند همه نقاط را دقیقاً توضیح دهد. مدل باید رابطه‌ای تقریبی پیدا کند که در مجموع مفید باشد. این نکته مهم است: یادگیری ماشین در بسیاری از کاربردها به دنبال تقریب مفید از یک رابطه پیچیده است، نه کشف یک قانون مطلق و بی‌خطا.

2.12. بازگشت به تقلب

در تشخیص تقلب می‌توان به‌طور کلی نوشت:

$$f_{\theta}(x_1, x_2, \dots, x_{100}) = \text{نمره تقلب}$$

x ها ویژگی‌های تراکنش‌اند و θ مجموعه پارامترهای مدل است. آموزش با استفاده از تراکنش‌های تاریخی این پارامترها را تنظیم می‌کند. در تولید، تراکنش جدید وارد همان تابع می‌شود و با پارامترهای آموخته‌شده نمره دریافت می‌کند.

3. شبکه عصبی از داخل چگونه کار می‌کند؟

مدل خطی تک‌ورودی به ما اجازه داد معنای پارامتر و یادگیری را بدون پیچیدگی اضافی ببینیم. اکنون می‌توان همان ایده را گسترش داد. در یک مسئله واقعی مانند تشخیص تقلب، تصمیم به یک ورودی وابسته نیست. ممکن است مبلغ، مکان، دستگاه، زمان، سابقه مشتری، تعداد تراکنش‌های اخیر و ده‌ها ویژگی دیگر هم‌زمان نقش داشته باشند.

نخست مدل را از یک ورودی به چند ورودی گسترش می‌دهیم:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

فرض کنید x_1 میزان غیرعادی بودن مبلغ، x_2 غیرعادی بودن مکان و x_3 میزان آشنایی دستگاه برای سامانه باشد. مقادیر ورودی را چنین فرض می‌کنیم:

$$x_1 = 0.8 \quad x_2 = 0.6 \quad x_3 = 0.2$$

و پارامترها چنین باشند:

$$w_1 = 1.1 \quad w_2 = 1.4 \quad w_3 = -0.9 \quad b = -0.7$$

محاسبه نتیجه:

$$z = 1.1 \times 0.8 + 1.4 \times 0.6 - 0.9 \times 0.2 - 0.7$$

$$z = 0.88 + 0.84 - 0.18 - 0.7 = 0.84$$

از این محاسبه می‌توان نقش وزن را دید. وزن مثبت باعث می‌شود افزایش ورودی در جهت افزایش z اثر بگذارد و وزن منفی می‌تواند اثر کاهنده داشته باشد. بزرگی وزن نیز میزان تأثیر آن ورودی در این محاسبه را تغییر می‌دهد. به همین دلیل گاهی گفته می‌شود «وزن اهمیت ورودی را نشان می‌دهد». این تعبیر برای شهود اولیه مفید است، اما در شبکه بزرگ تفسیر یک وزن منفرد به عنوان «اهمیت یک مفهوم» معمولاً درست نیست، زیرا اثر آن به سایر وزن‌ها، مقیاس ورودی، لایه‌های بعدی و فعال‌سازی‌ها وابسته است.

جمله ثابت نیز اجازه می‌دهد خروجی حتی زمانی که همه ورودی‌ها صفرند مقدار پایه‌ای داشته باشد. اگر x_1 ، x_2 و x_3 همگی صفر باشند، z برابر b خواهد بود. جمله ثابت مانند وزن معمولاً پارامتر آموختنی است.

3.1. نورون چگونه از همین محاسبه ساخته می‌شود؟

تا اینجا فقط جمع وزن‌دار ورودی‌ها را داریم. در شبکه عصبی معمولاً خروجی این جمع وارد تابع دیگری می‌شود که تابع فعال‌سازی نام دارد:

$$a = \varphi(z)$$

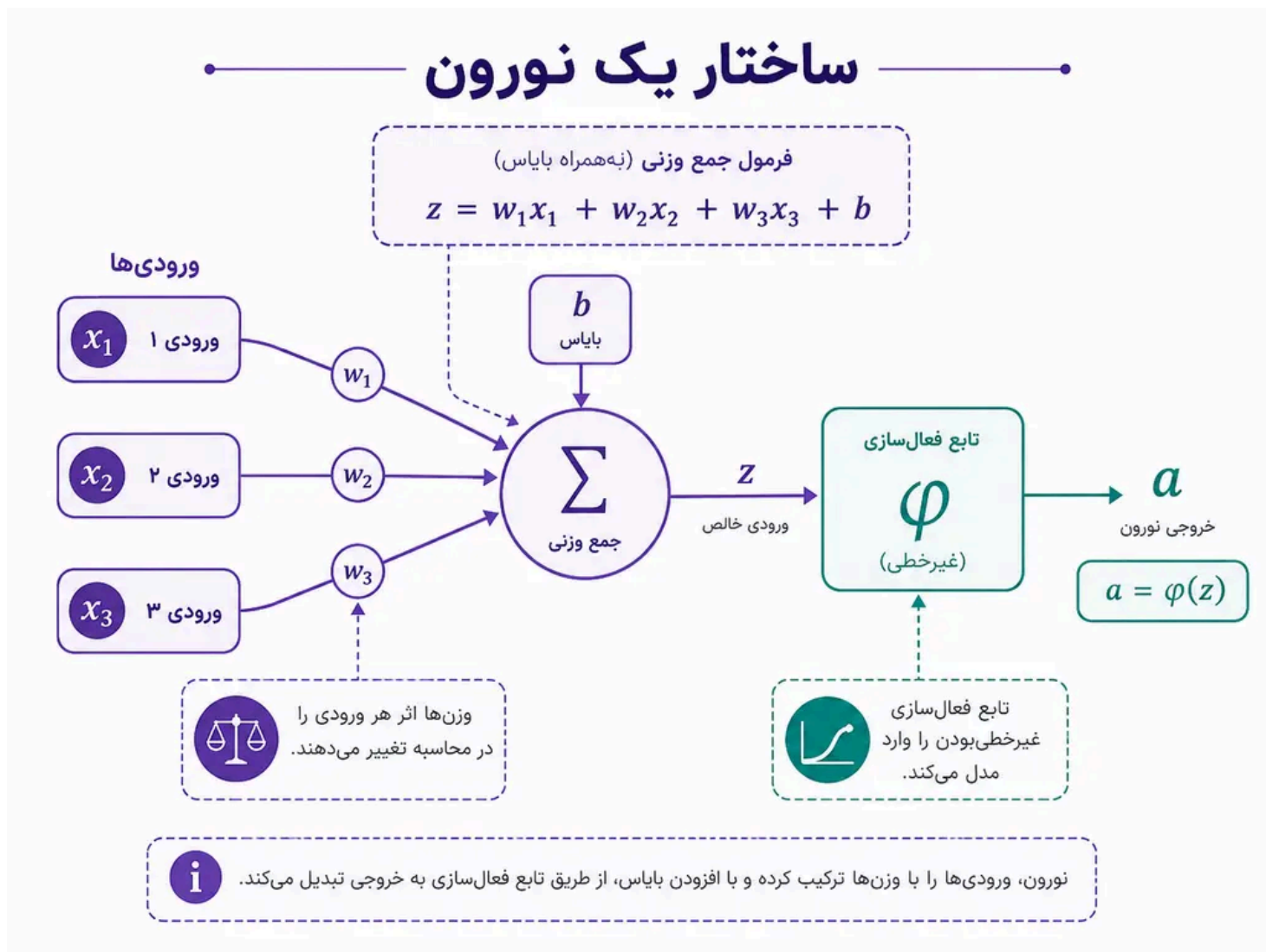
پس یک نورون ساده را می‌توان با دو مرحله توصیف کرد:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

$$a = \varphi(z)$$

این همان چیزی است که در نمودارهای آموزشی با یک دایره و چند پیکان نشان داده می‌شود. اگر فقط تصویر دایره‌ها را ببینیم، شبکه عصبی ممکن است رازآلود به نظر برسد؛ اما داخل نورون چیزی بسیار معمولی رخ می‌دهد: ضرب، جمع و اجرای یک تابع.

تشبیه نورون به سلول عصبی مغز از نظر تاریخی برای نام‌گذاری مفید بوده است، ولی شبکه عصبی مدرن را نباید شبیه‌سازی دقیق مغز دانست. برای درک مهندسی بهتر است آن را یک ساختار محاسباتی عددی ببینیم.



3.2. چرا تابع فعال‌سازی وجود دارد؟

اگر چند لایه فقط از تبدیل‌های خطی تشکیل شوند، اضافه کردن لایه قدرتی را که از شبکه عمیق انتظار داریم ایجاد نمی‌کند. فرض کنید لایه نخست $h = ax + b$ و لایه دوم $y = ch + d$ باشد. با جای‌گذاری h در رابطه دوم داریم:

$$y = c(ax + b) + d = cax + cb + d$$

می‌توان این رابطه را دوباره به شکل $y = Ax + B$ نوشت. یعنی ترکیب دو تبدیل خطی هنوز یک تبدیل خطی است. حتی اگر چندین لایه خطی پشت سر هم قرار گیرند، کل آن‌ها قابل کاهش به یک تبدیل خطی هستند.

تابع فعال‌سازی با وارد کردن غیرخطی بودن این محدودیت را می‌شکند. یکی از توابع ساده و رایج، یکسوساز خطی است:

$$\text{ReLU}(z) = \max(0, z)$$

اگر z برابر $3/7$ باشد، خروجی $3/7$ است و اگر z برابر $-2/1$ باشد، خروجی صفر می‌شود. تابع معروف دیگر سیگموئید است:

$$\sigma(z) = 1 / (1 + e^{-z})$$

سیگموئید هر عدد حقیقی را به بازه صفر تا یک می‌برد. برای $z = 0.84$ مقدار تقریبی آن 0.698 است. در برخی مسائل دسته‌بندی می‌توان خروجی را نمره یا در شرایط مناسب احتمال تفسیر کرد، اما هر عدد بین صفر و یک به‌طور خودکار یک احتمال کالبره شده نیست.

نیاز به غیرخطی بودن را می‌توان در تقلب دید. اثر مبلغ همیشه خطی نیست. مبلغ 5000 برای یک مشتری عادی می‌تواند غیرمعمول باشد ولی برای مشتری سازمانی روزمره باشد. مکان جدید ممکن است فقط همراه دستگاه جدید مهم شود. این تعامل‌ها را یک رابطه خطی ساده به‌خوبی نمایش نمی‌دهد.

3.3. لایه چپست و بازنمایی چگونه تغییر می‌کند؟

اگر چند نورون کنار هم قرار گیرند یک لایه داریم. با دو ورودی و سه نورون می‌توان نوشت:

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

$$z_3 = w_{31}x_1 + w_{32}x_2 + b_3$$

بعد از اعمال تابع فعال‌سازی، a_1 ، a_2 و a_3 ساخته می‌شوند. بنابراین بردار ورودی $[x_1, x_2]$ به بردار جدید $[a_1, a_2, a_3]$ تبدیل می‌شود. این خروجی‌ها می‌توانند ورودی لایه بعدی باشند. در این نگاه هر لایه یک بازنمایی را به بازنمایی دیگری تبدیل می‌کند.

لایه ورودی محل ورود ویژگی‌هاست. لایه‌های میان ورودی و خروجی «پنهان» نامیده می‌شوند، نه به این دلیل که ماهیت رازآلود دارند، بلکه چون خروجی آن‌ها خروجی نهایی مدل نیست. لایه خروجی نتیجه موردنیاز مسئله را تولید می‌کند. وجود چندین لایه محاسباتی سبب می‌شود از اصطلاح شبکه عصبی عمیق استفاده شود.

3.4. محاسبه عددی یک گذر رو به جلو

برای اینکه شبکه عصبی را یک موجود اسرارآمیز تصور نکنیم، یک شبکه کوچک را از ابتدا تا انتها حساب می‌کنیم. دو ورودی داریم:

$$x_1 = 1 \quad x_2 = 2$$

لایه پنهان دو نورون دارد. برای نورون اول وزن‌ها و جمله ثابت چنین‌اند:

$$w_{11} = 0.5 \quad w_{12} = 0.2 \quad b_1 = -0.1$$

محاسبه نورون اول:

$$z_1 = 0.5 \times 1 + 0.2 \times 2 - 0.1 = 0.8$$

$$h_1 = \text{ReLU}(0.8) = 0.8$$

برای نورون دوم داریم:

$$w_{21} = -0.3 \quad w_{22} = 0.8 \quad b_2 = 0$$

$$z_2 = -0.3 \times 1 + 0.8 \times 2 = 1.3$$

$$h_2 = \text{ReLU}(1.3) = 1.3$$

پس لایه پنهان ورودی $[1, 2]$ را به بازنمایی $[0.8, 1.3]$ تبدیل کرده است. حالا نورون خروجی این دو مقدار را دریافت می‌کند. وزن‌های خروجی را 1.2 و -0.5 و جمله ثابت را 0.2 فرض می‌کنیم:

$$z = 0.2 + 1.3 \times 0.5 - 0.8 \times 1.2$$

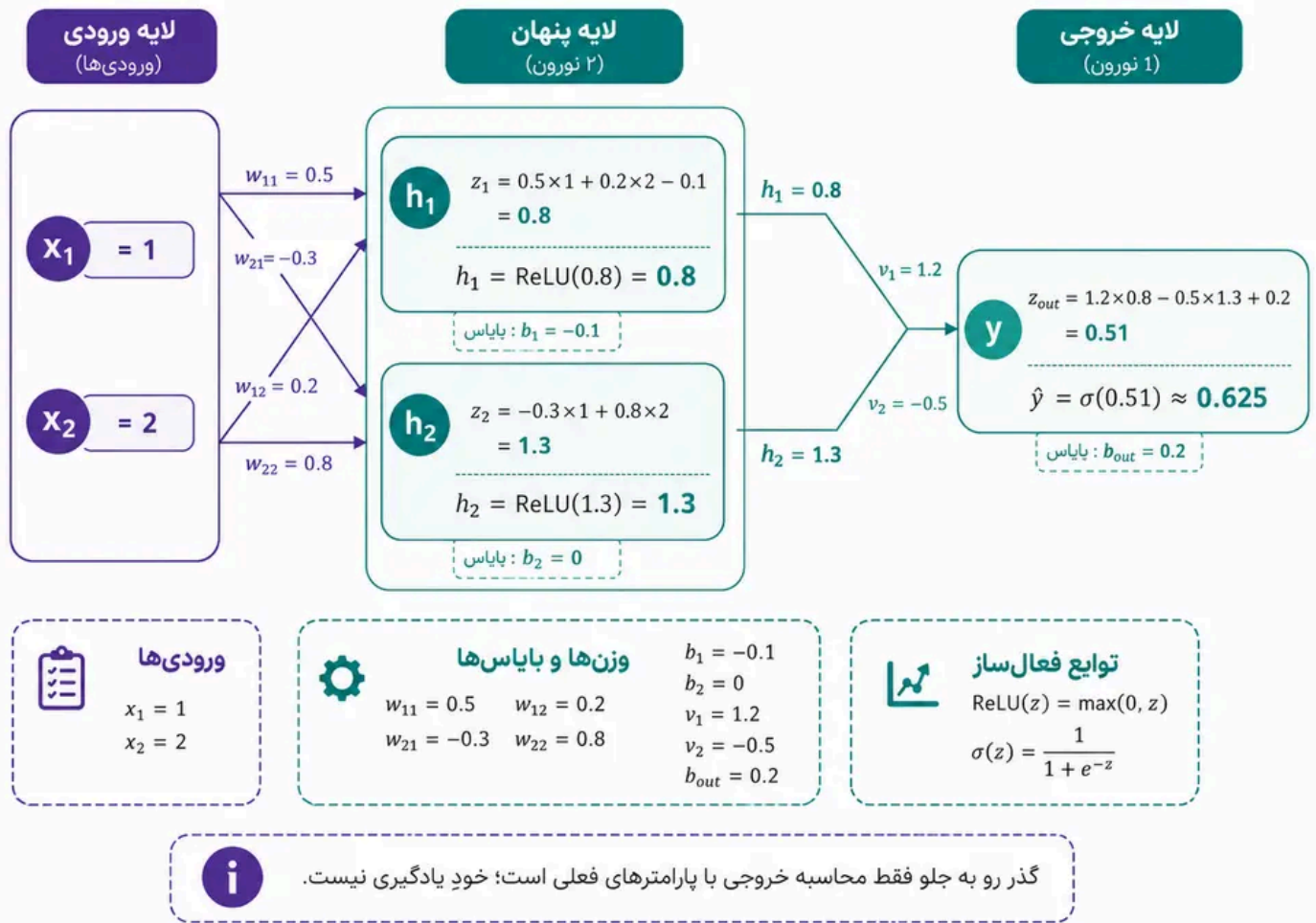
$$z = 0.2 + 0.65 - 0.96 = 0.51$$

اگر روی خروجی سیگموید اعمال شود:

$$\hat{y} = \sigma(0.51) \approx 0.625$$

بنابراین شبکه ورودی $[1, 2]$ را گرفته و خروجی تقریباً ۰/۶۲۵ تولید کرده است. تمام این فرایند «گذر رو به جلو» است. در این مرحله هیچ یادگیری رخ نداده؛ فقط پارامترهای فعلی برای محاسبه خروجی استفاده شده‌اند. همین‌گذر در آموزش نیز اجرا می‌شود، اما آموزش پس از تولید خروجی مراحل دیگری دارد.

گذر رو به جلو در یک شبکه کوچک



3.6. شبکه کوچک چند پارامتر دارد؟

نورون نخست لایه پنهان دو وزن و یک جمله ثابت دارد؛ نورون دوم نیز سه پارامتر دارد. پس لایه پنهان شش پارامتر دارد. نورون خروجی نیز دو وزن و یک جمله ثابت دارد، یعنی سه پارامتر دیگر. در مجموع شبکه ۹ پارامتر دارد.

این همان مفهوم پایه‌ای است که در عبارت «مدل هفت میلیارد پارامتر دارد» نیز وجود دارد. تفاوت فقط مقیاس و پیچیدگی معماری است. در عمل وزن‌ها به صورت تنسور و ماتریس نگهداری می‌شوند. اگر یک لایه هزار ورودی و دو هزار خروجی داشته باشد، ماتریس وزن آن دو میلیون مقدار دارد و بردار جمله ثابت دو هزار مقدار دیگر اضافه می‌کند.

3.7. پارامترهای مدل کجا هستند؟

پس از آموزش، وزن‌ها در فایل‌های مدل یا نقاط ذخیره ثبت می‌شوند. هنگام استنتاج این داده‌های عددی در حافظه اصلی یا حافظه شتاب‌دهنده مانند پردازنده گرافیکی بارگذاری می‌شوند. بنابراین تعداد پارامتر فقط یک عدد تبلیغاتی نیست و مستقیماً بر فضای ذخیره‌سازی، حافظه و هزینه محاسبه اثر می‌گذارد.

اگر صرفاً برای ساختن شهود فرض کنیم هر پارامتر چهار بایت مصرف کند، یک میلیارد پارامتر نزدیک چهار گیگابایت فضا برای وزن‌ها می‌خواهد. در عمل از دقت‌های عددی مختلف و روش‌های فشرده‌سازی مانند کمّی‌سازی استفاده می‌شود، اما اصل ارتباط میان تعداد پارامتر و منابع فیزیکی پابرجاست.

3.8. دانش مدل چگونه با پارامترها مرتبط است؟

عبارت «دانش داخل وزن‌ها ذخیره شده است» اگر به‌عنوان میان‌بر زبانی استفاده شود قابل فهم است، اما نباید آن را عینی و رکوردی تصور کرد. معمولاً نمی‌توان یک وزن خاص را پیدا کرد و گفت این وزن عبارت «تهران پایتخت ایران است» را نگه می‌دارد. رفتار آموخته‌شده از تعامل تعداد زیادی پارامتر، معماری، فعال‌سازی‌ها و ورودی فعلی به وجود می‌آید.

در مدل قیمت خانه، از بیرون می‌گفتیم مدل آموخته است با افزایش متراژ قیمت نیز معمولاً افزایش پیدا می‌کند. داخل مدل این «دانش» به شکل جمله فارسی وجود نداشت؛ مقدار مثبت وزن سبب می‌شد افزایش $\times$ خروجی را افزایش دهد. در شبکه بزرگ نیز باید میان توصیف انسانی قابلیت و سازوکار عددی داخلی تفاوت قائل شد.

3.9. بازنمایی، عمق و ظرفیت

لایه‌های پنهان می‌توانند بازنمایی ورودی را مرحله‌به‌مرحله تغییر دهند. در تصویر، لایه‌های نخست ممکن است نسبت به الگوهای ساده‌تر حساس شوند و لایه‌های بعدی ترکیب‌های پیچیده‌تری از آن‌ها بسازند. در تشخیص تقلب نیز مدل می‌تواند ترکیب‌هایی از ویژگی‌ها ایجاد کند که برای دسته‌بندی مفیدند، حتی اگر انسان برای آن ترکیب‌ها نام ساده‌ای نداشته باشد.

افزایش ظرفیت مدل به آن اجازه می‌دهد توابع پیچیده‌تری را نمایش دهد، اما ظرفیت بیشتر به‌تنهایی کیفیت را تضمین نمی‌کند. داده بد، هدف آموزشی نامناسب یا معماری نادرست می‌تواند یک مدل بسیار بزرگ اما ضعیف ایجاد کند. بنابراین تعداد پارامتر را نباید معیار مستقیم هوشمندی در نظر گرفت.

3.10 اثر معماری این موضوع

در نرم‌افزار کلاسیک اغلب بسته اجرایی و پیکربندی عناصر اصلی استقرارند. در سامانه یادگیری ماشین، وزن‌های مدل نیز یک بسته مستقل و نسخه‌پذیر هستند. ممکن است تصویر کانتینر و کد سرویس در دو محیط یکسان باشد اما نسخه مدل فرق کند و خروجی تغییر کند. بنابراین مدیریت نسخه مدل، ذخیره‌سازی، بارگذاری در حافظه، سازگاری با کد سرویس و ردگیری مدل مستقرشده بخشی از معماری عملیاتی می‌شود.

4. آموزش واقعاً چه می‌کند و داده چگونه رفتار مدل را شکل می‌دهد؟

اکنون تصویر گذر رو به جلو را داریم. می‌دانیم ورودی وارد شبکه می‌شود، محاسبات لایه به لایه انجام می‌شوند و خروجی ساخته می‌شود. اما پرسش اساسی هنوز باقی است: اگر مدل پاسخ بدی تولید کند، چگونه می‌فهمد کدام وزن باید تغییر کند، در چه جهتی و به چه اندازه؟

پاسخ کامل این پرسش به بعد تعلق دارد؛ زیرا برای فهم دقیق آن باید گرادیان، پس‌انتشار، نزول گرادیانی، دسته، دوره و نرخ یادگیری را جداگانه بررسی کرد. در این نوشته فقط تا جایی پیش می‌رویم که فرایند یادگیری قابل فهم شود.

4.1. از پیش‌بینی تا زیان

فرض کنید مدل تقلب برای یک تراکنش خروجی 0.82 تولید کرده است، اما برچسب واقعی نشان می‌دهد تراکنش سالم بوده است. در این لحظه مدل فقط با پارامترهای فعلی گذر رو به جلو انجام داده و پاسخ داده است. برای یادگیری باید بتوانیم بگوییم این پاسخ چقدر نامناسب بوده است.

تابع زیان همین نقش را دارد. در رگرسیون ممکن است میانگین مجذور خطا استفاده شود و در دسته‌بندی توابع دیگری مانند آنتروپی متقاطع رایج باشند. جزئیات ریاضی آن‌ها در اینجا هدف اصلی نیست؛ مهم این است که آموزش به یک عدد نیاز دارد تا بتواند کیفیت مجموعه پارامترهای فعلی را ارزیابی کند.

4.2. گرادیان چه اطلاعاتی می‌دهد؟

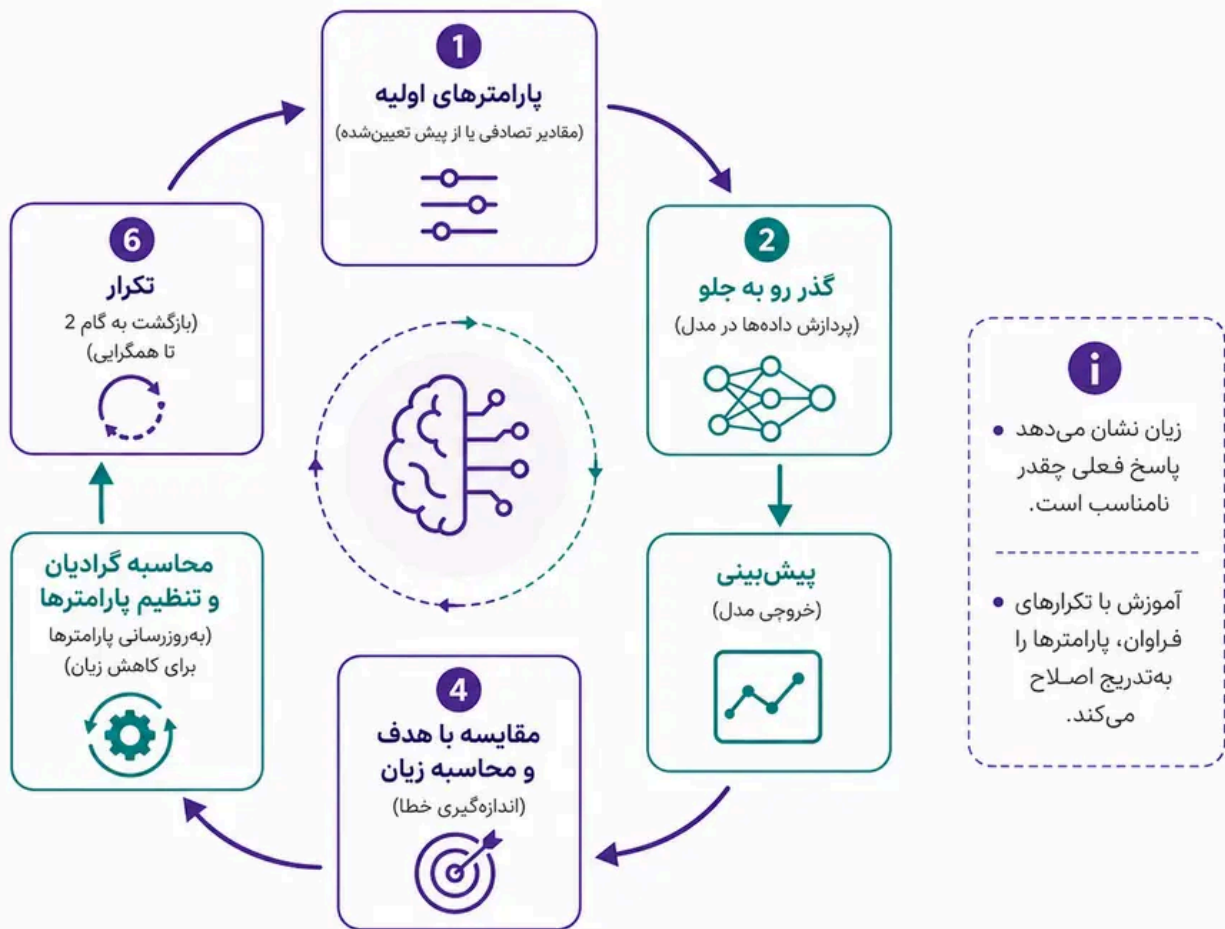
اگر فقط بدانیم زیان بالا است، هنوز نمی‌دانیم کدام پارامتر را چگونه تغییر دهیم. گرادیان اطلاعاتی فراهم می‌کند درباره اینکه تغییر کوچک پارامترها چه اثری بر زیان خواهد داشت. تشبیه رایج، حرکت روی دامنه کوه است: اگر هدف رسیدن به نقطه کم‌ارتفاع باشد، شیب به ما می‌گوید در چه جهتی حرکت کنیم. این تشبیه فقط برای شهود است؛ فضای پارامترهای یک شبکه بزرگ ممکن است میلیاردها بُعد داشته باشد.

پس‌انتشار با استفاده از قاعده زنجیره‌ای مشتق‌ها اثر زیان را از خروجی به لایه‌های قبلی منتقل می‌کند و گرادیان پارامترهای مختلف را محاسبه می‌کند. سپس الگوریتم بهینه‌سازی مانند نزول گرادیانی بر اساس این گرادیان‌ها وزن‌ها و جمله‌های ثابت را اندکی تغییر می‌دهد.

پارامترهای اولیه ← پیش‌بینی ← مقایسه با پاسخ مطلوب ← زیان ← گرادیان ← تنظیم پارامترها ← تکرار

در یک مدل واقعی این چرخه روی نمونه‌های بسیار زیاد و بارها تکرار می‌شود. یادگیری معمولاً حاصل مجموعه بزرگی از به‌روزرسانی‌های کوچک است، نه یک جهش ناگهانی از «نمی‌داند» به «می‌داند».

چرخه آموزش مدل



4.3. داده چگونه رفتار آینده را شکل می‌دهد؟

مدل فقط سیگنال‌هایی را می‌بیند که داده در اختیارش قرار می‌دهد. اگر الگوی مهمی در داده حضور نداشته باشد، مدل اطلاعات کافی برای یادگیری آن ندارد. اگر داده سوگیری داشته باشد، پارامترها نیز بر همان اساس شکل می‌گیرند. اگر برچسب‌ها غلط باشند، فرایند آموزش مدل را به سمت هدف نادرست هل می‌دهد.

برای مثال اگر بانک در گذشته نوع خاصی از تقلب را به درستی کشف نکرده باشد، بخشی از تراکنش‌های متقلبانه در داده تاریخی با برچسب «سالم» وجود خواهند داشت. الگوریتم آموزش ممکن است کاملاً درست پیاده‌سازی شده باشد، اما از پاسخ‌های نادرست یاد می‌گیرد. بنابراین کیفیت مدل را نمی‌توان جدا از کیفیت داده بررسی کرد.

4.4. نمونه آموزشی، ویژگی و برچسب

در مثال قیمت خانه، هر ردیف مجموعه داده یک نمونه آموزشی بود. در تشخیص تقلب، هر تراکنش یک نمونه آموزشی است. ورودی مدل مجموعه‌ای از ویژگی‌هاست؛ مانند مبلغ، ساعت، مکان، نوع دستگاه، تعداد تراکنش‌های چند دقیقه اخیر و رفتار تاریخی مشتری.

ویژگی لزوماً یک ستون خام پایگاه داده نیست. از زمان دقیق تراکنش می‌توان «ساعت روز»، «روز تعطیل بودن» یا «فاصله زمانی از تراکنش قبلی» را استخراج کرد. در یادگیری عمیق بخشی از ساخت بازنمایی نیز می‌تواند داخل خود مدل انجام شود، اما مفهوم ویژگی همچنان به اطلاعاتی اشاره دارد که برای پیش‌بینی استفاده می‌شوند.

در یادگیری نظارت‌شده هر نمونه علاوه بر ورودی، مقدار هدف یا برچسب دارد. در تشخیص هرزنامه برچسب «هرزنامه/ سالم»، در دسته‌بندی تصویر «گربه/سگ» و در قیمت خانه مقدار واقعی فروش است.

4.5. یادگیری نظارت‌شده

در یادگیری نظارت‌شده مجموعه داده را می‌توان به صورت زوج‌های (x, y) دید. مدل تلاش می‌کند رابطه‌ای پیدا کند که از x به y برسد. مزیت این روش روشن بودن هدف است، اما هزینه برچسب‌گذاری می‌تواند بسیار زیاد باشد. در تصویر پزشکی ممکن است هر نمونه به پزشک متخصص نیاز داشته باشد. در تشخیص تقلب نیز نتیجه واقعی برخی تراکنش‌ها تنها پس از تحقیق مشخص می‌شود.

4.6. یادگیری بدون نظارت

در یادگیری بدون نظارت برچسب صریحی مانند «تقلب/سالم» وجود ندارد. ممکن است هدف کشف ساختار، گروه‌ها یا الگوهای پنهان در داده باشد. برای مثال، می‌توان مشتریان را بر اساس رفتار مالی به خوشه‌هایی تقسیم کرد بدون آنکه از قبل نام بخش‌ها مشخص شده باشد.

نبود برچسب به معنی نبود هدف یا الگوریتم نیست. الگوریتم هنوز مسئله ریاضی مشخصی دارد؛ فقط سیگنال آموزشی از نوع دیگری است.

4.7. یادگیری خودنظارتی؛ چرا برای زبان مهم است؟

حال تصور کنید می‌خواهیم یک مدل زبانی را روی میلیاردها جمله آموزش دهیم. اگر برای هر جمله انسان مجبور باشد پاسخ درست را برچسب بزند، هزینه عملی ناممکن خواهد شد. اما متن ویژگی مهمی دارد: می‌توان هدف را از خود متن استخراج کرد.

جمله: «تهران پایتخت ایران است.»

می‌توان بخشی از جمله را به عنوان ورودی و بخش بعدی را به عنوان هدف در نظر گرفت. برای نمونه:

ورودی: «تهران پایتخت ایران ...» هدف: «است»

ورودی: «من امروز به دانشگاه ...» هدف: «رفتم»

انسان برای این نمونه‌ها برچسب تازه‌ای ننوشته است؛ خود داده هدف را ایجاد کرده است. این ایده یکی از شکل‌های یادگیری خودنظارتی است. مدل با تعداد بسیار زیادی از چنین پیش‌بینی‌ها مواجه می‌شود و برای کاهش خطا ناچار می‌شود الگوهای فراوانی از زبان، ساختار جمله، هم‌نشینی واژه‌ها، روابط مفهومی و برخی الگوهای واقعی جهان را در پارامترهای خود منعکس کند.

در این نوشته وارد این بحث نمی‌شویم که چگونه قابلیت‌های پیچیده از وظیفه پیش‌بینی بخش بعدی متن پدیدار می‌شوند؛ این موضوع به نوشته‌های مربوط به مدل زبانی و ترنسفورمر تعلق دارد که شاید در آینده‌ای نچندان دور منتشرم کنیم. هدف فعلی فقط درک این نکته است که حجم عظیمی از متن می‌تواند بدون برچسب‌گذاری دستی تک‌به‌تک، سیگنال آموزشی تولید کند.

سه نوع اصلی یادگیری



یادگیری نظارت‌شده

مدل با داده‌های برچسب‌دار آموزش می‌بیند تا از ورودی، برچسب درست را پیش‌بینی کند.

مثال‌ها



ایمیل → هرزنامه / سالم



خانه → قیمت



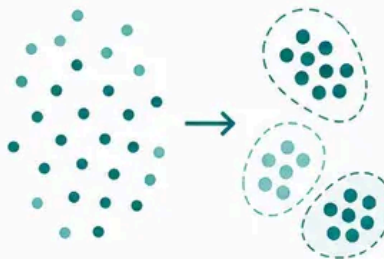
داده آموزشی شامل ورودی و برچسب است.



یادگیری بدون نظارت

مدل فقط ورودی‌ها را می‌بیند و باید ساختار یا الگوهای پنهان را کشف کند.

مثال



هدف، کشف ساختار یا الگو در داده بدون برچسب صریح است.



یادگیری خودنظارتی

مدل از خود داده برای ساختن هدف آموزشی استفاده می‌کند و بدون برچسب بیرونی آموزش می‌بیند.

مثال

جمله اصلی

تهران پایتخت ایران است



جمله ورودی (بخشی مخفی شده)

تهران پایتخت ایران [MASK]

هدف مدل

است



هدف آموزشی از خود داده ساخته می‌شود.



یادگیری خودنظارتی برای آموزش مدل‌های زبانی بزرگ اهمیت بنیادی دارد.

در همین‌جا باید از دو افراط پرهیز کرد. یک افراط می‌گوید مدل زبانی فقط متن اینترنت را مانند پایگاه داده حفظ کرده است. افراط دیگر می‌گوید مدل اصلاً چیزی را حفظ نمی‌کند و فقط مفاهیم عمومی می‌آموزد. در عمل تعمیم و حفظ کردن می‌توانند هر دو رخ دهند و نسبت آن‌ها به داده، مدل و آموزش وابسته است.

کد آموزش + داده آموزشی + مدل اولیه ← آموزش ← مدل آموزش‌دیده

مدل آموزش‌دیده + ورودی جدید ← استنتاج ← پیش‌بینی

آموزش ممکن است روی زیرساخت محاسباتی سنگین انجام شود، در حالی که استنتاج روی زیرساخت دیگری برای پاسخ به درخواست‌های تولید اجرا شود. در آموزش علاوه بر گذر رو به جلو، زیان، پس‌انتشار و به‌روزرسانی پارامترها وجود دارد. در استنتاج معمولاً پارامترها ثابت‌اند و فقط پاسخ محاسبه می‌شود.

4.8. چرا فراخوانی مدل زبانی معمولاً آموزش نیست؟

وقتی از یک مدل زبانی از طریق رابط برنامه‌نویسی استفاده می‌کنید، معمولاً فرایند آموزش مدل پایه اجرا نمی‌شود. پیام شما وارد زمینه زمان اجرا می‌شود و مدل با پارامترهای موجود پاسخ تولید می‌کند. اگر در ابتدای گفت‌وگو گفته شود «نام محصول ما آریا است» و چند پیام بعد مدل آن را به یاد داشته باشد، این رفتار به‌خودی‌خود نشان‌دهنده تغییر وزن‌ها نیست؛ اطلاعات در زمینه گفت‌وگو وجود داشته است.

همین تفکیک بعداً برای بازیابی بسیار مهم می‌شود. می‌توان اطلاعات جدید را هنگام استنتاج در اختیار مدل گذاشت، بدون اینکه آن اطلاعات در پارامترها آموزش داده شوند. بنابراین «استفاده از دانش در زمینه» و «یادگیری پارامتری» دو چیز متفاوت‌اند.

4.9. چرخه کامل آموزش مدل تقلب

یک میلیون تراکنش تاریخی آماده شده است. برای هر تراکنش ویژگی‌ها استخراج شده و برچسب نهایی وجود دارد. مدل با پارامترهای اولیه شروع می‌کند. تراکنش‌ها وارد مدل می‌شوند، گذر رو به جلو انجام می‌شود، پیش‌بینی با برچسب مقایسه می‌شود، زیان محاسبه می‌شود، پس‌انتشار گرادین‌ها را به دست می‌آورد و بهینه‌ساز پارامترها را کمی تغییر می‌دهد. این روند بارها تکرار می‌شود تا مدل روی هدف آموزش رفتار بهتری پیدا کند. سپس وزن‌ها ذخیره می‌شوند و در تولید برای تراکنش‌های جدید استفاده می‌شوند.

اما همین‌جا پرسش تازه‌ای شکل می‌گیرد: اگر مدل تمام داده آموزشی را عالی پاسخ دهد، آیا واقعاً مدل خوبی است؟ پاسخ لزوماً مثبت نیست، زیرا هدف اصلی فقط یادگیری گذشته نیست؛ باید روی داده ندیده نیز عملکرد مناسب داشته باشد.

5. آموزش در برابر استنتاج و مسئله تعمیم

تا اینجا دو تصویر ساخته‌ایم: آموزش، که در آن پارامترها تغییر می‌کنند، و استنتاج، که در آن مدل آموزش‌دیده روی ورودی جدید اجرا می‌شود. اکنون باید این دو را دقیق‌تر کنار هم قرار دهیم و بعد به یکی از بنیادی‌ترین مفاهیم یادگیری ماشین برسیم: تعمیم.

5.1. آموزش و استنتاج هدف یکسانی ندارند

فرض کنید مدل تقلب با ده میلیون تراکنش تاریخی آموزش دیده است. در طول آموزش، تراکنش‌ها بارها وارد مدل شده‌اند، مدل پیش‌بینی ساخته، پاسخ با برچسب واقعی مقایسه شده و پارامترها تغییر کرده‌اند. در پایان یک بسته مدل داریم که وزن‌های آموخته‌شده در آن ذخیره شده‌اند.

اکنون مدل در محیط عملیاتی مستقر می‌شود. تراکنش جدید می‌آید، ویژگی‌ها ساخته می‌شوند و مدل نمره ۰/۸۷ تولید می‌کند. هدف دیگر یادگیری نیست؛ هدف استفاده از چیزی است که قبلاً یاد گرفته شده است.

به صورت خلاصه، آموزش می‌پرسد «پارامترها چه مقادیری باید داشته باشند؟» و استنتاج می‌پرسد «با پارامترهای فعلی، این ورودی چه خروجی‌ای دارد؟».

5.2. مقایسه فنی آموزش و استنتاج

در آموزش، داده تاریخی و در مسائل نظارت‌شده برچسب وارد فرایند می‌شوند. گذر رو به جلو، محاسبه زیان، محاسبه گرادینان، پس‌انتشار و به‌روزرسانی وزن‌ها انجام می‌شود. در استنتاج، معمولاً فقط گذر رو به جلو برای ورودی جدید لازم است و پارامترها تغییر نمی‌کنند.

به همین دلیل آموزش معمولاً از نظر حافظه و توان محاسباتی سنگین‌تر است. در آموزش باید اطلاعات میانی لازم برای پس‌انتشار نیز نگهداری شوند. در مقابل یک استنتاج منفرد ساده‌تر است، هرچند اگر میلیون‌ها درخواست در روز وجود داشته باشد مجموع هزینه استنتاج می‌تواند بسیار زیاد شود.

تناوب نیز متفاوت است. آموزش ممکن است هر هفته یا هر ماه انجام شود، اما استنتاج برای هر تراکنش یا درخواست تولید رخ می‌دهد. آموزش می‌تواند چند ساعت طول بکشد، در حالی که سرویس استنتاج ممکن است نیازمند تأخیر زیر چند صد میلی‌ثانیه باشد. این تفاوت باعث می‌شود معماری آموزش و معماری خدمت‌دهی مدل الزامات متفاوتی داشته باشند.

آموزش در برابر استنتاج

آموزش	
هدف: یادگیری پارامترها	
ورودی: داده تاریخی و در بسیاری مسائل برچسب	
خروجی: مدل آموزش دیده	
تغییر پارامترها: بله	
هزینه محاسباتی: زیاد	
زیرساخت: معمولاً سنگین تر	

استنتاج	
هدف: تولید پاسخ برای ورودی جدید	
ورودی: داده جدید	
خروجی: پیش بینی یا پاسخ	
تغییر پارامترها: معمولاً خیر	
هزینه محاسباتی: کمتر از آموزش برای هر اجرا	
کاربرد: پاسخ گویی در تولید	

وقتی از یک مدل زبانی از طریق API استفاده می کنیم، معمولاً در حال استنتاج هستیم، نه آموزش.

5.3 استفاده از مدل زبانی از طریق رابط معمولاً استنتاج است

اگر یک مدل پایه بزرگ قبلاً توسط فراهم کننده آموزش داده شده باشد و شما از طریق رابط از آن بخواهید متنی را خلاصه کند، در حالت معمول وزن های اصلی مدل تغییر نمی کنند. پیام به مدل داده می شود، گذر رو به جلو و فرایند تولید اجرا می شود و پاسخ برمی گردد. پس فراخوانی رابط با آموزش برابر نیست.

اگر مدل در همان گفت و گو اطلاعاتی را که چند پیام قبل گفته اید به کار ببرد، ممکن است علت آن وجود اطلاعات در زمینه باشد، نه تغییر پارامترها. این تمایز در آینده برای فهم تنظیم دقیق، بازیابی و مهندسی زمینه حیاتی است.

5.4. تعمیم؛ هدف اصلی آموزش

حال فرض کنید مدل روی تمام داده های آموزشی تقریباً بدون خطا پاسخ می دهد. آیا می توان نتیجه گرفت مدل بسیار خوب است؟ خیر. هدف یادگیری ماشین معمولاً این نیست که نمونه های گذشته را بازتولید کند؛ قرار است روی نمونه هایی که قبلاً ندیده نیز عملکرد مفید داشته باشد. این توانایی «تعمیم» نام دارد.

در مثال خانه، اگر مدل روی مترهای ۵۰، ۷۰، ۹۰، ۱۱۰ و ۱۳۰ آموزش دیده و برای خانه ۱۰۰ متری نیز قیمت معقولی پیش‌بینی می‌کند، نشانه‌ای از تعمیم داریم. مدل صرفاً یک جدول جست‌وجو نیست؛ رابطه‌ای را آموخته که به نقطه جدید نیز قابل اعمال است.

5.5. بیش‌برازش؛ وقتی مدل گذشته را بیش از حد جدی می‌گیرد

فرض کنید مجموعه داده کوچک است و یکی از خانه‌ها به دلیل شرایط خانوادگی خاص با قیمتی غیرمعمول فروخته شده است. اگر مدل آن قدر پیچیده باشد که برای عبور دقیق از تک‌تک نقاط، از جمله آن استثنا، شکل بسیار پیچیده‌ای پیدا کند، ممکن است خطای آموزش تقریباً صفر شود اما روی خانه‌های جدید عملکرد بدی داشته باشد. این حالت بیش‌برازش است.

بیش‌برازش را نباید فقط با «حفظ کردن رکوردها» یکی دانست. مدل می‌تواند به الگوهای اتفاقی یا غیرعمومی داده بیش از حد تکیه کند. نمونه مشهور آن مجموعه تصاویری است که سگ‌ها بیشتر روی چمن و گربه‌ها بیشتر داخل خانه‌اند. مدل ممکن است بخشی از تصمیم را بر پس‌زمینه بنا کند و هنگام دیدن سگ داخل خانه اشتباه کند.

خطای کم روی آموزش $\neq$ تعمیم خوب

5.6. کم‌برازش؛ وقتی مدل ظرفیت یا ساختار کافی ندارد

در سوی دیگر، ممکن است مدل از اساس برای مسئله بیش از حد ساده باشد. اگر تقلب را فقط از روی مبلغ با یک تابع خطی پیش‌بینی کنیم در حالی که الگوی واقعی به تعامل مکان، دستگاه، زمان و سابقه مشتری وابسته است، مدل حتی روی داده آموزشی نیز نمی‌تواند الگوی اصلی را به‌خوبی بگیرد. این کم‌برازش است.

هدف یافتن مدلی است که نه آن قدر ساده باشد که الگو را از دست بدهد و نه آن قدر به جزئیات مجموعه آموزش بچسبد که روی آینده شکست بخورد. در یادگیری عمیق مدرن رابطه‌ی ظرفیت و تعمیم پیچیده‌تر از یک نمودار ساده است، اما اصل نیاز به تعمیم همچنان پابرجاست.

5.7. چرا داده را به آموزش، اعتبارسنجی و آزمون تقسیم می‌کنیم؟

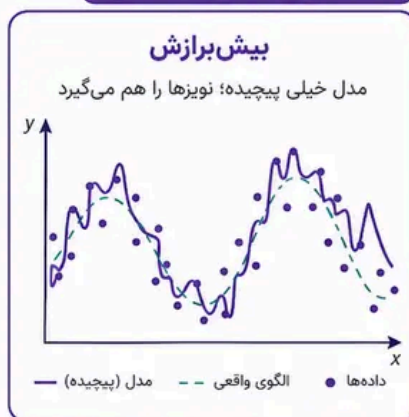
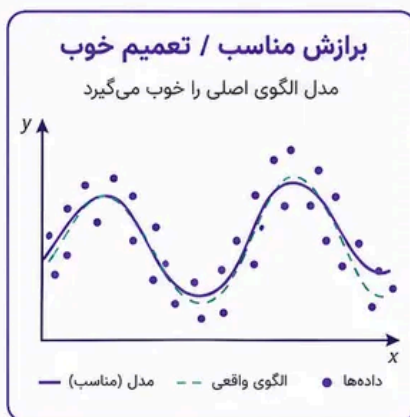
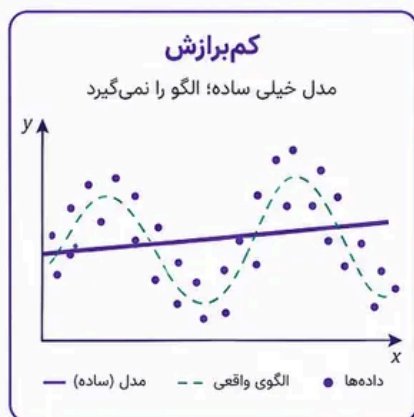
اگر همان داده‌ای که مدل با آن آموزش دیده برای قضاوت نهایی استفاده شود، نمی‌توان فهمید مدل واقعاً روی داده جدید چه وضعیتی دارد. فرض کنید یک میلیون تراکنش تاریخی داریم. می‌توان مثلاً هشتصد هزار مورد را برای آموزش، صد هزار مورد را برای اعتبارسنجی و صد هزار مورد را برای آزمون کنار گذاشت. این اعداد فقط مثال‌اند؛ روش تقسیم به ماهیت داده بستگی دارد.

مجموعه آموزش مستقیماً در به‌روزرسانی پارامترها نقش دارد. مجموعه اعتبارسنجی برای تصمیم‌های توسعه مانند انتخاب مدل یا ابرپارامتر استفاده می‌شود. مجموعه آزمون تا حد امکان باید از فرایند تصمیم‌گیری دور بماند تا در پایان برآورد نسبتاً بی‌طرفانه‌ای از عملکرد روی داده ندیده فراهم کند.

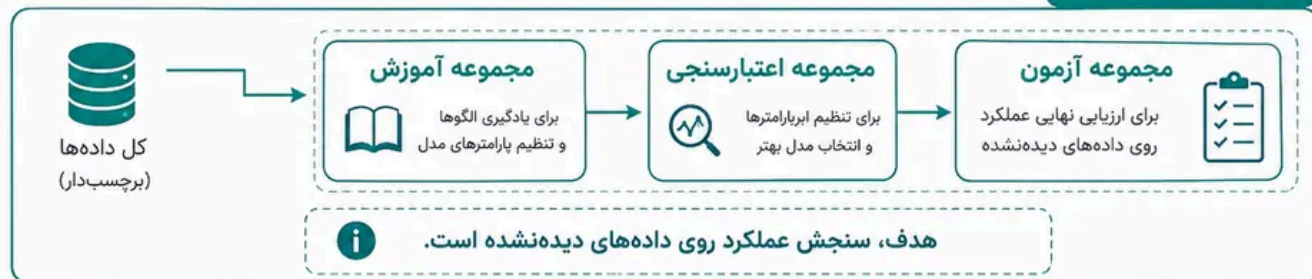
اگر بارها نتیجه آزمون را نگاه کنیم و بر اساس آن مدل را تغییر دهیم، توسعه به آزمون نیز سازگار می‌شود و آزمون استقلال خود را از دست می‌دهد. این شبیه دانشجویی است که بارها سؤال‌های امتحان نهایی را قبل از امتحان می‌بیند.

تعمیم، بیش‌برازش و کم‌برازش

(A) انطباق مدل با داده‌ها



(B) تقسیم داده‌ها



کم بودن خطای آموزش به‌تنهایی به معنی مدل خوب نیست.

5.8. رابطه با آزمون نرم‌افزار کلاسیک

در نرم‌افزار تعیین‌گر معمولاً انتظار دقیق برای یک ورودی تعریف می‌شود. برای مثال: موجودی ۱۰۰ است، برداشت ۱۲۰ درخواست می‌شود، نتیجه باید رد باشد. اگر سیستم اجازه دهد، خطا روشن است. اما مدل تقلب ممکن است روی صد هزار نمونه برخی تقلب‌ها را درست تشخیص دهد، برخی را از دست بدهد و تعدادی تراکنش سالم را اشتباه علامت‌گذاری کند. ارزیابی مدل بنابراین به رفتار آماری روی یک مجموعه نماینده نگاه می‌کند.

این تفاوت به معنای حذف آزمون کلاسیک نیست. بخش‌های قطعی مانند مجوز، قرارداد رابط، ساختار پاسخ، تراکنش پایگاه داده و سیاست‌های مالی همچنان به آزمون‌های صریح نیاز دارند. خود مدل علاوه بر این‌ها به ارزیابی آماری نیاز دارد. یک سامانه هوش مصنوعی واقعی هر دو نوع اطمینان را می‌خواهد.

5.9. میان‌برهای آماری و تغییر الگو

در تشخیص تقلب فرض کنید بیشتر نمونه‌های تقلب آموزش مربوط به یک پذیرنده خاص باشند. مدل ممکن است به رابطه سطحی «این پذیرنده = تقلب» تکیه کند و روی داده آموزش بسیار خوب ظاهر شود. اگر متقلبان پذیرنده را عوض کنند، مدل ناگهان ضعیف می‌شود. این نمونه‌ای از یادگیری میان‌بر است: مدل چیزی را پیدا کرده که برای داده گذشته مفید بوده، اما ماهیت عمومی مسئله را به‌خوبی نگرفته است.

5.10. نشت داده

یکی از خطرهای مهم، نشت اطلاعات از آینده یا از بخش آزمون به آموزش است. برای مثال ممکن است ویژگی‌ای در مدل تقلب استفاده شود که در زمان واقعی تصمیم‌گیری هنوز در دسترس نیست و فقط بعد از بررسی نهایی تولید می‌شود. اگر چنین ویژگی‌ای ناخواسته وارد آموزش شود، مدل در آزمون نتایج عالی می‌دهد ولی در تولید نمی‌تواند همان اطلاعات را داشته باشد.

همچنین اگر نمونه‌های بسیار مشابه یک مشتری هم در آموزش و هم در آزمون قرار گیرند، مدل ممکن است از شباهت استفاده کند و نتیجه آزمون بیش از حد خوش‌بینانه شود. به همین دلیل تقسیم تصادفی ساده همیشه کافی نیست؛ ساختار داده و شرایط تولید باید در طراحی تقسیم در نظر گرفته شوند.

5.11. پیام معماری تعمیم

هیچ مجموعه آزمونی نمی‌تواند تمام آینده را پوشش دهد. بنابراین استقرار مدل همیشه نوعی ریسک آماری دارد. از همین‌جا پایش، بازبینی انسانی، آستانه تصمیم و مسیر جایگزین معنا پیدا می‌کنند. اگر مدل هرگز اشتباه نمی‌کرد، بسیاری از این کنترل‌ها لازم نبودند.

اگر مدل عدد ۰/۹۳ تولید کند نیز نباید بدون بررسی گفت «مدل ۹۳ درصد مطمئن است». تفسیر چنین عددی به نوع مدل، هدف آموزش و کالیبراسیون بستگی دارد. ممکن است صرفاً یک نمره باشد. در آینده و مفاهیم ارزیابی، این تفاوت با دقت بیشتری بررسی خواهد شد.

6. وقتی مدل وارد معماری می‌شود، «درست بودن» دیگر فقط یک مقدار صفر و یک نیست

تا اینجا تقریباً تمام اجزای فنی لازم را ساخته‌ایم. اکنون باید از سطح خود مدل یک لایه بالاتر برویم و ببینیم ورود یک مؤلفه یادگرفته‌شده و آماری به سامانه چه فرض‌هایی را در معماری نرم‌افزار تغییر می‌دهد. این بخش برای معمار نرم‌افزار اهمیت ویژه دارد، زیرا مسئله اصلی فقط انتخاب مدل نیست؛ باید مدل را در کنار بخش‌های قطعی و قابل ممیزی سیستم قرار داد.

6.1. تعیین‌گرایی دقیقاً چیست؟

فرض کنید در سامانه بانکی قاعده‌ای وجود دارد که اگر موجودی قابل برداشت کمتر از مبلغ انتقال باشد، تراکنش رد شود. اگر موجودی ۱۰ میلیون و مبلغ انتقال ۱۲ میلیون باشد، پاسخ روشن است. با ورودی و وضعیت یکسان، سیستم باید همان تصمیم را تکرار کند.

یک تابع تعیین‌گرا با ورودی و حالت یکسان خروجی یکسان می‌دهد. اما تعیین‌گرا بودن را نباید با درست بودن اشتباه گرفت. تابعی که مالیات را همیشه ۵۰ درصد محاسبه می‌کند ممکن است کاملاً تعیین‌گرا و در عین حال کاملاً غلط باشد. بنابراین تعیین‌گرایی درباره تکرارپذیری و صحت درباره انطباق با نیازمندی یا واقعیت است.

6.2. مدل آماری الزاماً اجرای تصادفی ندارد

یک شبکه عصبی با وزن‌های ثابت و ورودی یکسان می‌تواند هر بار دقیقاً همان عدد را تولید کند. پس خود گذر رو به جلو ممکن است تعیین‌گرا باشد. آنچه آماری است رابطه‌ای است که مدل برای مسئله آموخته و احتمال خطای آن روی نمونه‌های جدید. در مدل‌های مولد لایه دیگری نیز وجود دارد: روش تولید می‌تواند از نمونه‌برداری استفاده کند و در نتیجه حتی با ورودی و وزن یکسان خروجی‌های متفاوتی بسازد.

پس جمله «هوش مصنوعی غیرقطعی است» بیش از حد کلی است. معمار باید بپرسد عدم قطعیت دقیقاً از کجا می‌آید: از ماهیت آماری مدل، از تفسیر نمره، از نمونه‌برداری زمان تولید، از داده متغیر یا از زمینه متفاوت؟

6.3. خطای مدل و باگ نرم‌افزار

فرض کنید مدل برای یک تراکنش واقعی متقلبانه نمره ۰/۱۸ داده و سیستم آن را عبور داده است. آیا مدل باگ دارد؟ پاسخ لزوماً مانند نرم‌افزار کلاسیک نیست. اگر محاسبات ماتریسی اشتباه پیاده‌سازی شده باشند یا سرویس تبدیل ارز واحد را غلط تبدیل کند، باگ نرم‌افزاری داریم. اما اگر همه محاسبات درست انجام شده باشند و مدل یک نمونه را اشتباه دسته‌بندی کند، این می‌تواند خطای آماری مدل باشد و بخشی از نرخ خطای مورد انتظار آن محسوب شود.

این تمایز برای تحلیل ریشه‌ای مشکل مهم است. از نگاه کاربر همه موارد ممکن است به جمله «سیستم اشتباه کرد» ختم شوند، اما معمار باید میان خرابی سرویس، خطای داده، خطای ویژگی، خطای مدل، تغییر توزیع و خطای سیاست تصمیم تفاوت بگذارد.

6.4. شکست معنایی در مدل‌های مولد

در سامانه مبتنی بر مدل زبانی ممکن است سرویس از نظر عملیاتی کاملاً سالم باشد؛ وضعیت شبکه موفق است، ساختار پاسخ معتبر است، تأخیر مناسب است و هیچ استثنایی رخ نداده؛ اما محتوای پاسخ از نظر واقعیت غلط باشد. در چنین حالتی کنترل‌های سنتی سلامت سرویس همه سبز هستند، ولی از دید معنایی محصول شکست خورده است. این نوع شکست یکی از دلایلی است که ارزیابی و پایش سامانه هوش مصنوعی را از پایش صرف زیرساخت جدا می‌کند.

6.5. آستانه؛ تبدیل پیش‌بینی به تصمیم

فرض کنید مدل تقلب نمره ۰/۸۱ تولید می‌کند. هسته بانکی نمی‌تواند با «۰/۸۱» به تنهایی کاری انجام دهد؛ باید عملی مانند اجازه، توقف یا ارجاع به بازبینی انتخاب شود. اینجا لایه تصمیم وارد می‌شود:

اگر نمره تقلب $0.8 \leq$ باشد ← تراکنش نگه داشته شود

۰/۸ آستانه است. اگر مدل برای همان تراکنش ۰/۷۹ تولید کند و آستانه ۰/۸ باشد، تصمیم ممکن است اجازه باشد. اگر بدون تغییر مدل آستانه به ۰/۷۵ کاهش یابد، همان نمره به تصمیم متفاوتی منتهی می‌شود. بنابراین خروجی مدل و تصمیم کسب‌وکار یک چیز نیستند.

پیش‌بینی $\neq$ تصمیم

انتخاب آستانه نیز فقط یک مسئله فنی نیست. کاهش آستانه ممکن است تقلب بیشتری را بگیرد ولی تراکنش سالم بیشتری را هم اشتباه متوقف کند. افزایش آستانه می‌تواند هشدارهای غلط را کم کند اما تقلب بیشتری را از دست بدهد. هزینه هر نوع خطا به کسب‌وکار، قانون و تجربه مشتری وابسته است.

6.6. بازبینی انسانی چرا می‌تواند بخشی دائمی از معماری باشد؟

گاهی گفته می‌شود «فعلاً انسان را در حلقه می‌گذاریم تا مدل بهتر شود و بعد حذفش می‌کنیم». این مسیر در بعضی محصولات ممکن است درست باشد، اما در حوزه‌های پرریسک بازبینی انسانی می‌تواند کنترل دائمی باشد. معماری می‌تواند ناحیه‌های ریسک متفاوت تعریف کند:

ریسک > 0.2 ← تأیید خودکار

0.2 ≥ ریسک > 0.8 ← بازبینی انسانی

ریسک ≤ 0.8 ← نگه داشتن تراکنش

اعداد نمونه‌اند. نکته آن است که مدل الزاماً صاحب آخرین اختیار نیست. ممکن است نقش مدل امتیازدهی یا تصمیم‌یاری باشد و اختیار نهایی در برخی نواحی به انسان یا سیاست قطعی سپرده شود.

6.7. مسیر جایگزین و کاهش کنترل‌شده خدمت

اگر سرویس مدل از دسترس خارج شود، آیا کل پردازش تراکنش باید متوقف شود؟ پاسخ به حساسیت کسب‌وکار بستگی دارد. ممکن است موتور قواعد قدیمی به‌عنوان مسیر جایگزین فعال شود، تراکنش‌های خاص به بازبینی انسانی بروند یا فقط عملیات پرخطر محدود شوند. اصل مهم این است که خرابی مؤلفه هوش مصنوعی الزاماً به خرابی کامل محصول تبدیل نشود.

مسیر جایگزین فقط برای قطع سرویس نیست. داده ناقص، ورودی خارج از محدوده معمول، نمره نامعتبر یا شرایطی که مدل برای آن آموزش مناسب ندیده است نیز می‌توانند سبب فعال شدن مسیر امن‌تری شوند.

6.8. پایش سامانه هوش مصنوعی فراتر از پردازنده و حافظه است

در یک سرویس معمولی مصرف پردازنده، حافظه، تأخیر، تعداد خطا و دسترس‌پذیری اندازه‌گیری می‌شوند. همه این‌ها برای سرویس مدل نیز لازم‌اند، اما کافی نیستند. تصور کنید سرویس مدل ۹۹/۹۹ درصد در دسترس است و میانگین تأخیر ۴۰ میلی‌ثانیه دارد، اما الگوی تقلب طی سه ماه تغییر کرده و کیفیت تشخیص به‌شدت افت کرده است. از دید زیرساخت سرویس سالم و از دید کسب‌وکار معیوب است.

پس باید رفتار مدل نیز پایش شود: توزیع ورودی‌ها، نرخ علامت‌گذاری، تغییر میانگین نمره، عملکرد واقعی پس از رسیدن نتیجه نهایی و سایر نشانه‌های رفتاری. بررسی مفاهیم رانش داده و پایش مدل در حوزه این نوشته نیستند.

6.9. چه چیزهایی نباید به مدل واگذار شوند؟

اگر مقررات می‌گویند انتقال به حساب موجود در فهرست تحریم ممنوع است، این الزام احتمالی نیست. مدل نباید بگوید «با احتمال ۹۴ درصد باید مسدود شود». همین منطق برای کنترل دسترسی برقرار است. اگر کاربر مجوز مشاهده حسابی را ندارد، مدل نباید درباره اجازة او حدس بزند. سازگاری دفتر کل مالی، محدودیت‌های پایگاه داده، مجوزها و بسیاری از

کنترل‌های امنیتی و قانونی باید تعیین‌گرا بمانند.

مسئولیت مهم معمار آن است که مشخص کند کدام تصمیم برای رفتار یادگرفته‌شده مناسب است، کدام تصمیم باید قاعده صریح بماند و کجا مدل فقط نقش تصمیم‌یار داشته باشد.

6.10. خروجی مدل را حقیقت دامنه نکنید

اگر مدل نمره تقلب ۹۱/۰ تولید کند، طراحی خطرناکی است که بلافاصله در دامنه ثبت شود «این تراکنش قطعاً تقلبی است». نمره مدل یک ارزیابی است و حقیقت نهایی ممکن است بعداً با تحقیق مشخص شود. معماری بهتر این مفاهیم را جدا نگه می‌دارد.

ارزیابی-ریسک:

نسخه-مدل

نمره-مدل

زمان-ارزیابی

نسخه-سیاست

تصمیم-سیستم

نتیجه-بازبینی-انسانی

حقیقت-نهایی

این جداسازی برای ممیزی و ردیابی مهم است. اگر شش ماه بعد مشتری اعتراض کند، باید بتوان گفت کدام نسخه مدل استفاده شده، چه نمره‌ای تولید شده، کدام سیاست تصمیم‌گیری فعال بوده و نتیجه بازبینی چه بوده است.

6.11. یک معماری ساده اما بالغ برای تشخیص تقلب

تراکنش ابتدا وارد لایه قطعی می‌شود. احراز هویت، مجوز، موجودی، وضعیت حساب و قواعد قانونی بررسی می‌شوند. اگر تراکنش در همین مرحله غیرمجاز باشد، مدل نباید این قانون را حدس بزند. اگر از کنترل‌ها عبور کرد، ویژگی‌های مدل ساخته می‌شوند و مدل نمره ریسک می‌دهد. سیاست تصمیم بر اساس نمره مشخص می‌کند تراکنش تأیید، نگه داشته یا برای بازبینی ارسال شود. اگر مدل در دسترس نباشد، مسیر جایگزین فعال می‌شود. تمام این مراحل ثبت می‌شوند و حقیقت نهایی بعداً می‌تواند برای پایش و آموزش آینده استفاده شود.

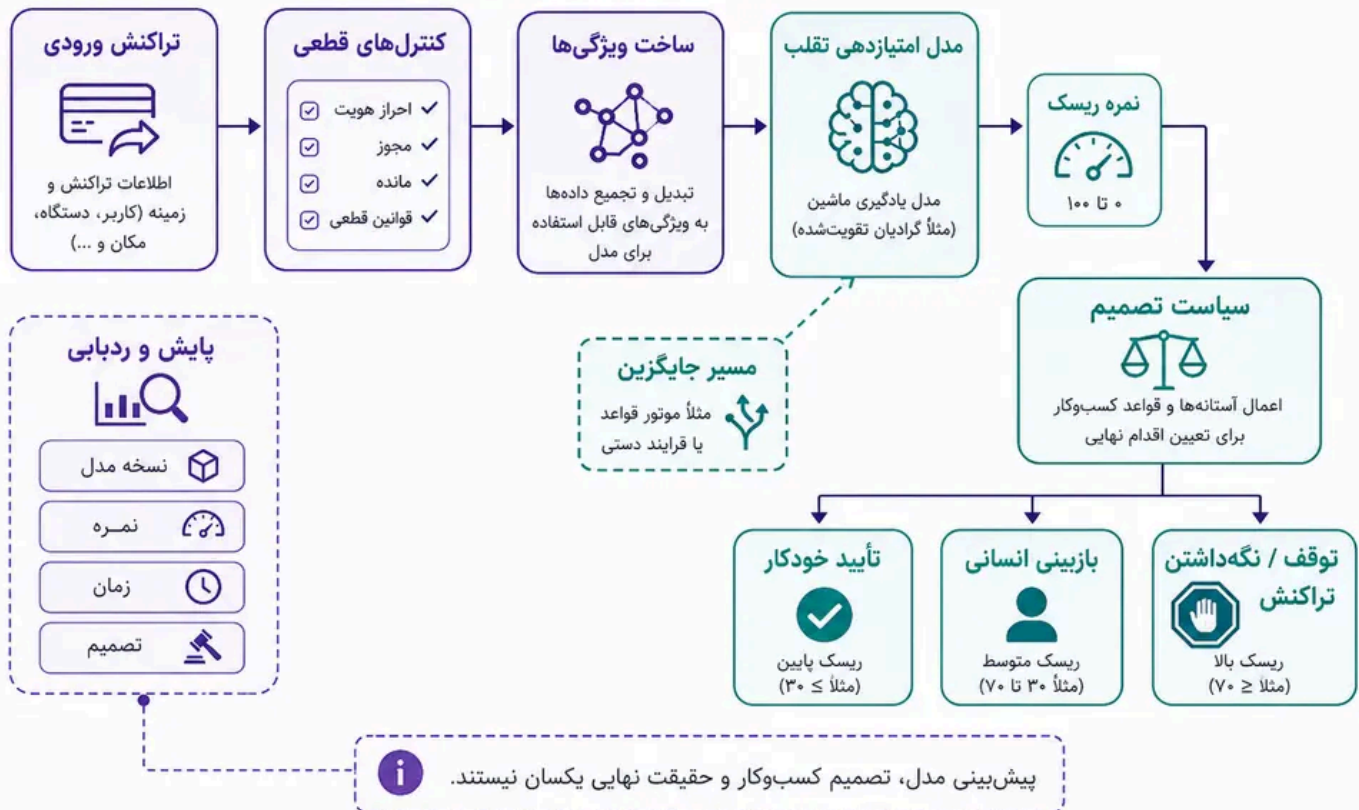
این معماری مدل را مهم می‌داند اما مدل را کل سامانه نمی‌داند. همین تفاوت میان یک استفاده نمایشی هوش مصنوعی و یک سامانه عملیاتی قابل اعتماد است.

جای مدل در معماری یک سامانه واقعی



سامانه کشف تقلب در تراکنش‌ها

(مثال واقعی)



6.12. چرا در هوش مصنوعی مولد مسئله سخت‌تر می‌شود؟

در یک دسته‌بندی ساده خروجی محدود است، مثلاً «تقلب/سالم». اما مدل زبانی می‌تواند پاراگراف آزاد تولید کند. یک پاسخ می‌تواند از نظر دستور زبان درست اما از نظر واقعیت غلط باشد؛ ممکن است واقعیت را درست بیان کند اما سیاست امنیتی را نقض کند؛ ممکن است مفهوم درست باشد اما قالب خروجی خراب باشد؛ یا اطلاعات مجاز کاربر دیگری را افشا کند. فضای شکست بسیار گسترده‌تر می‌شود و به همین دلیل لایه‌های کنترل بیشتری پیرامون مدل لازم است.

از سوی دیگر، درست نیست نتیجه بگیریم چون مدل آماری است هیچ تصمیم مهمی نباید به آن سپرده شود. سامانه‌های یادگیری ماشین امروز تصمیم‌های بسیار مهمی می‌گیرند. نکته این است که اعتمادپذیری آن‌ها باید متناسب با ماهیت آماری‌شان مهندسی شود. اطمینان‌بخشی برای تابع مالیاتی و مدل تقلب یک روش یکسان ندارد.

7. از داده تا مدل و از مدل تا یک سامانه هوش مصنوعی واقعی

در شش بخش گذشته مفاهیم را به ترتیب ساخته ایم. اکنون زمان آن است که همه آن‌ها را در یک زنجیره واحد کنار هم قرار دهیم. این بخش قرار نیست صرفاً تعریف‌ها را تکرار کند؛ هدف آن است که مسیر کامل از تعریف مسئله تا استفاده از مدل در تولید دیده شود و مرزهای مفهومی که در نوشته ساخته شده‌اند تثبیت شوند.

۱. پیش از مدل، مسئله و طراحی یادگیری وجود دارد

بانک ابتدا مسئله‌ای کسب‌وکاری دارد: شناسایی تراکنش‌های مشکوک. این جمله هنوز مسئله یادگیری ماشین دقیق نیست. باید مشخص شود مدل چه چیزی دریافت می‌کند، خروجی مطلوب چیست، برچسب معتبر چگونه تعریف می‌شود، کدام داده نماینده است و تصمیم نهایی چگونه از خروجی مدل ساخته می‌شود. بنابراین مدل خودش مسئله را تعریف نمی‌کند؛ انسان فضای یادگیری را طراحی می‌کند.

۲. داده، تجربه‌ای است که مدل در اختیار دارد

اگر ده میلیون تراکنش تاریخی داریم، کیفیت، پوشش و توزیع همین تراکنش‌ها بر رفتار آینده مدل اثر می‌گذارد. اگر نوعی تقلب در گذشته بسیار کم دیده شده یا برچسب‌گذاری غلط بوده باشد، آموزش سیگنال مناسبی دریافت نمی‌کند. مدل از حقیقت مطلق جهان آموزش نمی‌بیند؛ از داده‌ای آموزش می‌بیند که ما جمع‌آوری و آماده کرده‌ایم.

۳. مدل را با رابطه عمومی $\hat{y} = f\theta(x)$ ببینید

$$\hat{y} = f\theta(x)$$

x ورودی، θ مجموعه پارامترها و f ساختار محاسباتی است. معماری شکل f را تعیین می‌کند، آموزش θ را تنظیم می‌کند و استنتاج x جدید را وارد تابع می‌کند. همین تصویر از رگرسیون خطی ساده تا شبکه‌های بسیار بزرگ قابل استفاده است.

۴. آموزش یعنی تغییر سازمان‌یافته پارامترها

مدل با پارامترهای اولیه شروع می‌کند. ورودی وارد می‌شود، گذر رو به جلو پیش‌بینی را می‌سازد، پاسخ با هدف مقایسه می‌شود، زیان محاسبه می‌شود، گرادیان مشخص می‌کند چه تغییراتی مفیدند و پارامترها به‌روز می‌شوند. این چرخه بارها تکرار می‌شود. جمله انسانی «مدل تقلب را یاد گرفت» در سطح محاسباتی به این معناست که پارامترها به وضعیتی رسیده‌اند که تابع حاصل روی هدف موردنظر رفتار مفیدتری نشان می‌دهد.

یادگیری الزاماً به تولید قواعد صریح تبدیل نمی‌شود. وزن منفرد نیز معمولاً یک مفهوم یا واقعیت منفرد را نگه نمی‌دارد. قابلیت مدل از تعامل تعداد بسیار زیادی پارامتر در ساختار محاسباتی شکل می‌گیرد.

۵. در پایان آموزش یک بسته مدل واقعی داریم

وزن‌ها در فایل‌های مدل یا نقاط ذخیره نگهداری می‌شوند. این بسته می‌تواند چندین گیکابایت یا بیشتر باشد و در زمان استنتاج باید در حافظه مناسب بارگذاری شود. مدل نهایی با مجموعه داده یکی نیست؛ داده برای شکل دادن پارامترها استفاده شده است. مدل روی ورودی جدید محاسبه انجام می‌دهد و صرفاً رکوردی را از پایگاه داده پیدا نمی‌کند.

۶. استنتاج یعنی استفاده از چیزی که قبلاً آموخته شده است

در تولید پارامترها ثابت‌اند و ورودی جدید وارد مدل می‌شود. این نکته توضیح می‌دهد چرا استفاده از یک مدل زبانی از طریق رابط معمولاً آموزش نیست. زمینه می‌تواند اطلاعات تازه در اختیار مدل بگذارد بدون اینکه وزن‌ها تغییر کنند. بعدها همین مرز برای فهم تفاوت آموزش، تنظیم دقیق، بازیابی و مهندسی زمینه اهمیت حیاتی خواهد داشت.

۷. آموزش زمانی مفید است که به آینده تعمیم دهد

مدلی که فقط داده آموزش را به خوبی جواب دهد ممکن است در تولید شکست بخورد. بیش‌برازش یعنی وابستگی بیش از حد به جزئیات گذشته و کم‌برازش یعنی ناتوانی در گرفتن الگوی اصلی. تقسیم داده به آموزش، اعتبارسنجی و آزمون تلاشی برای تخمین عملکرد روی نمونه‌های ندیده است.

۸. پیش‌بینی، تصمیم و حقیقت نهایی سه مفهوم جدا هستند

خروجی مدل $\neq$ تصمیم کسب‌وکار $\neq$ حقیقت نهایی

مدل نمره یا پیش‌بینی تولید می‌کند. سیاست تصمیم مشخص می‌کند با آن چه عملی انجام شود. حقیقت نهایی ممکن است بعداً از تحقیقات یا رویداد واقعی به دست آید. این تفکیک اجازه می‌دهد آستانه و سیاست بدون آموزش دوباره مدل تغییر کنند و ردیابی، ممیزی و بهبود مدل نیز دقیق‌تر شوند.

۹. جایگاه اصطلاحات در یک نقشه ذهنی واحد

هوش مصنوعی حوزه بزرگ‌تر است. یادگیری ماشین یکی از روش‌های ساخت رفتار هوشمندانه با استفاده از داده است. یادگیری عمیق خانواده‌ای از یادگیری ماشین مبتنی بر شبکه‌های عصبی عمیق است. هوش مصنوعی مولد به مدل‌هایی اشاره دارد که محتوای جدید تولید می‌کنند. مدل پایه مدلی بزرگ و از پیش آموزش‌دیده است که می‌تواند پایه چند کاربرد باشد. مدل زبانی بزرگ مدلی بزرگ برای زبان است. ترنسفورمر نیز یک معماری است و نباید با خود هوش مصنوعی یا یک مدل مشخص یکی دانسته شود.

هوش مصنوعی با یادگیری ماشین یکسان نیست؛ یادگیری ماشین زیرمجموعه‌ای از هوش مصنوعی است.

یادگیری ماشین با یادگیری عمیق یکسان نیست؛ یادگیری عمیق یکی از خانواده‌های یادگیری ماشین است.

معماری مدل با پارامترهای مدل یکی نیست؛ معماری ساختار محاسباتی را مشخص می‌کند و آموزش مقادیر پارامترها را پیدا می‌کند.

ورودی با پارامتر یکی نیست؛ ورودی در هر فراخوان تغییر می‌کند و پارامتر بخشی از مدل آموزش‌دیده است.

پارامتر با ابرپارامتر یکی نیست؛ پارامتر در آموزش یاد گرفته می‌شود و ابرپارامتر معمولاً نحوه طراحی یا آموزش را کنترل می‌کند.

آموزش با استنتاج یکی نیست؛ آموزش پارامترها را تغییر می‌دهد و استنتاج معمولاً از پارامترهای ثابت استفاده می‌کند.

گذر رو به جلو به تنهایی آموزش نیست؛ همان گذر در استنتاج نیز انجام می‌شود.

مدل با پایگاه داده یکی نیست؛ مدل برای ورودی محاسبه انجام می‌دهد و نباید صرفاً مخزن جست‌وجوی واقعیت‌ها تصور شود.

زیان کم روی مجموعه آموزش با مدل خوب برابر نیست؛ تعمیم روی داده ندیده معیار اساسی است.

پیش‌بینی با تصمیم یکی نیست؛ لایه سیاست می‌تواند روی خروجی مدل تصمیم متفاوتی بگیرد.

پیش‌بینی با حقیقت نهایی یکی نیست؛ مدل یک ارزیابی یا تخمین ارائه می‌کند.

زمینه زمان اجرا با آموزش یکی نیست؛ اطلاعاتی که در درخواست داده می‌شوند الزاماً پارامترهای مدل را تغییر نمی‌دهند.

۱۱. مهم‌ترین تغییر ذهنی برای معمار نرم‌افزار

در نرم‌افزار کلاسیک عادت داریم رفتار را تقریباً مستقیم به کد و پی‌کربندی نسبت دهیم. در سامانه هوش مصنوعی باید مدل کامل‌تری داشته باشیم:

رفتار سامانه = کد + قواعد صریح + معماری مدل + پارامترهای آموخته‌شده + داده آموزشی + ورودی زمان اجرا + سیاست تصمیم

در سامانه‌های مبتنی بر مدل زبانی بعداً زمینه، بازیابی، ابزارها و تنظیمات تولید نیز به این معادله ذهنی اضافه می‌شوند. نتیجه آن است که رفع اشکال چندبعدی می‌شود. خروجی غلط می‌تواند ناشی از باگ نرم‌افزار، داده نامناسب، مدل ضعیف، نسخه مدل اشتباه، آستانه نامناسب، تغییر توزیع یا سیاست تصمیم غلط باشد. معمار باید این لایه‌ها را از یکدیگر جدا ببیند.

هم‌زمان بخش‌های قطعی نرم‌افزار از بین نمی‌روند. کنترل دسترسی نباید حدس زده شود. محدودیت مالی و قواعد قانونی نباید تقریب زده شوند. سازگاری داده و تراکنش‌های حیاتی نباید به مدل احتمالاتی سپرده شوند. معماری بالغ هوش مصنوعی دقیقاً مرز میان رفتار آموخته‌شده و رفتار قطعی را طراحی می‌کند.

۱۲. «درست بودن» در سامانه هوش مصنوعی چگونه تغییر می‌کند؟

برای تابع قطعی ممکن است سؤال اصلی این باشد که آیا مطابق مشخصات کار می‌کند یا نه. برای مدل باید علاوه بر آن پرسید روی چه توزیعی عملکرد خوبی دارد، چه نوع خطاهایی می‌کند، هزینه هر خطا برای کسب‌وکار چیست، در شرایط عدم قطعیت چه اتفاقی می‌افتد، اگر مدل در دسترس نبود چه می‌کنیم و چگونه افت کیفیت رفتاری را تشخیص می‌دهیم. این پرسش‌ها پایه ارزیابی، پایش، ایمنی و حاکمیت در درس‌های آینده‌اند.

8. جمع‌بندی نهایی

این نوشته از یک پرسش آغاز شد: «وقتی گفته می‌شود مدل یاد گرفته است، دقیقاً چه چیزی تغییر کرده؟» پاسخ اکنون روشن‌تر است. یک مدل، ساختاری محاسباتی و پارامتردار است. آموزش با استفاده از داده و یک هدف عددی، پارامترهای آن را تغییر می‌دهد تا رفتار تابع نسبت به مسئله مطلوب‌تر شود. در مدل خطی این پارامترها w و b بودند؛ در شبکه عصبی همان ایده به میلیون‌ها یا میلیارد‌ها وزن و جمله ثابت گسترش پیدا می‌کند.

گذر رو به جلو ورودی را با پارامترهای فعلی به خروجی تبدیل می‌کند. آموزش علاوه بر گذر رو به جلو، زیان و گرادیان را برای تنظیم پارامترها به کار می‌گیرد. استنتاج از پارامترهای آموخته‌شده روی ورودی جدید استفاده می‌کند. داده فقط ماده خام نیست؛ تجربه‌ای است که رفتار مدل از آن شکل می‌گیرد. هدف واقعی نیز صرفاً کم کردن خطای گذشته نیست، بلکه تعمیم به داده ندیده است.

از دید معماری، مدل فقط یک مؤلفه در سامانه است. پیش‌بینی مدل با تصمیم کسب‌وکار و حقیقت نهایی یکسان نیست. کنترل‌های قطعی، آستانه، بازبینی انسانی، مسیر جایگزین، ثبت ممیزی و پایش همگی بخشی از طراحی سیستم واقعی‌اند. هوش مصنوعی جایگزین ساده برنامه‌نویسی کلاسیک نیست؛ نوع تازه‌ای از مؤلفه رفتاری را به سامانه اضافه می‌کند که باید با ماهیت آماری خودش مهندسی شود.

با این مدل ذهنی، اصطلاحاتی مانند پارامتر، وزن، آموزش، استنتاج، تعمیم و شبکه عصبی دیگر نام‌هایی جدا از هم نیستند، بلکه اجزای یک فرایند واحدند: داده وارد فرایند آموزش می‌شود، آموزش پارامترهای مدل را تنظیم می‌کند، بسته مدل ذخیره و مستقر می‌شود و استنتاج از همان پارامترها برای ورودی‌های جدید خروجی می‌سازد. این پایه برای درک عمیق‌تر سامانه‌های هوش مصنوعی ضروری است.

8.1. برای مطالعه بیشتر

برای مطالعه عمیق‌تر مبانی یادگیری ماشین، کتاب «یادگیری عمیق» نوشته ایان گودفلو، یوشوا بنجیو و آرون کوروئل منبع استاندارد است. فصل پنجم آن با عنوان مبانی یادگیری ماشین برای تعمیم، ظرفیت، بیش‌برازش و مفاهیم پایه و فصل ششم برای شبکه‌های پیش‌خور عمیق مناسب است.

کتاب «یادگیری عمیق: مبانی و مفاهیم» نوشته کریستوفر بیشاپ و هیو بیشاپ نیز منبعی مدرن و دقیق برای پایه مفهومی و ریاضی است. برای رویکرد دانشگاهی، یادداشت‌های دوره CS229 دانشگاه استنفورد در بخش‌های ابتدایی یادگیری نظارت‌شده و رگرسیون خطی ارزش مطالعه دارند. برای مرور کاربردی، دوره فشرده یادگیری ماشین گوگل نیز بخش‌های مناسبی درباره رگرسیون، زیان، تعمیم و شبکه عصبی ارائه می‌کند.

واژه‌نامه جامع فارسی-انگلیسی

هوش مصنوعی

Artificial Intelligence

یادگیری ماشین

Machine Learning

یادگیری عمیق

Deep Learning

هوش مصنوعی نمادین

Symbolic AI

سامانه خبره

Expert System

قاعده

Rule

موتور قواعد

Rule Engine

هوش مصنوعی مولد

Generative AI

مدل پایه

Foundation Model

مدل زبانی بزرگ

Large Language Model

مدل

Model

معماری مدل

Model Architecture

تابع

Function

تابع پارامتردار

Parameterized Function

پارامتر

Parameter

ابرپارامتر

Hyperparameter

وزن

Weight

جمله ثابت

Bias

ورودی

Input

خروجی

Output	پیش بینی
Prediction	
	آموزش
Training	
	استنتاج
Inference	
	الگوریتم آموزش
Training Algorithm	
	تابع زیان
Loss Function	
	میانگین مجذور خطا
Mean Squared Error	
	گرادیان / بردار شیب
Gradient	
	پس انتشار
Backpropagation	
	نزول گرادیانی
Gradient Descent	
	نرخ یادگیری

Learning Rate	شبکه عصبی
Neural Network	نورون
Neuron	تابع فعال سازی
Activation Function	یکسوساز خطی اصلاح شده
ReLU	سیگموید
Sigmoid	لایه
Layer	لایه ورودی
Input Layer	لایه پنهان
Hidden Layer	لایه خروجی
Output Layer	شبکه عصبی عمیق

Deep Neural Network

گذر رو به جلو

Forward Pass

بازنمایی

Representation

تانسور

Tensor

ضرب ماتریسی

Matrix Multiplication

نقطه ذخیره

Checkpoint

نمونه آموزشی

Training Example

ویژگی

Feature

برچسب

Label

یادگیری نظارت شده

Supervised Learning

یادگیری بدون نظارت

Unsupervised Learning

یادگیری خودنظارتی

Self-Supervised Learning

سیگنال آموزشی

Training Signal

مجموعه آموزش

Training Set

مجموعه اعتبارسنجی

Validation Set

مجموعه آزمون

Test Set

تعمیم

Generalization

بیش برآزش

Overfitting

کم برآزش

Underfitting

نشت داده

Data Leakage

توزیع داده

Data Distribution

یادگیری میان‌بر

Shortcut Learning

ارزیابی مدل

Model Evaluation

تعیین‌گرا / قطعی

Deterministic

احتمالاتی

Probabilistic

صحت

Correctness

شکست معنایی

Semantic Failure

آستانه

Threshold

بازبینی انسانی

Human Review

مسیر جایگزین

Fallback

کاهش کنترل‌شده خدمت

Graceful Degradation

پایش

Monitoring

ردیابی

Traceability

ممیزی

Audit

حقیقت نهایی

Ground Truth

تصمیم کسب و کار

Business Decision

زمینه

Context

زمان اجرا

Runtime

استقرار

Deployment

بسته مدل

Model Artifact

پایگاه داده

Database

رابط برنامه نویسی کاربردی

API

پیکربندی

Configuration

کالیبراسیون

Calibration

مشاهده گزارش

گزارش سالیانه جامعه برنامه نویسی

تصویری واقعی از برنامه نویسان ایران؛ مسیر یادگیری، فناوری، شغل و آینده